



Rearrangement Distance Problems: An updated survey

ANDRE RODRIGUES OLIVEIRA, Faculdade de Computação e Informática, Universidade Presbiteriana Mackenzie, São Paulo, Brazil

KLAIRTON LIMA BRITO, ALEXSANDRO OLIVEIRA ALEXANDRINO, and GABRIEL SIQUEIRA, Instituto de Computação, Universidade Estadual de Campinas (UNICAMP), Campinas, Brazil

ULISSES DIAS, Faculdade de Tecnologia, Universidade Estadual de Campinas (UNICAMP), Limeira, Brazil

ZANONI DIAS, Instituto de Computação, Universidade Estadual de Campinas (UNICAMP), Campinas, Brazil

One of the challenges in the Comparative Genomics field is to infer how close two organisms are based on the similarities and differences between their genetic materials. Recent advances in DNA sequencing have made complete genomes increasingly available. That said, several new algorithms trying to infer the distance between two organisms based on genome rearrangements have been proposed in the literature. However, given the diversity of approaches, the diversity of genome rearrangement events, or even how each work models the genomes and what assumptions are made by each of them, finding the ideal algorithm for each situation or simply knowing the range of applicable approaches can be challenging. In this work, we review these approaches having the algorithmic and combinatorial advances since 2010 as our main focus. This survey aims to organize the recently published papers using a concise notation and to indicate the gaps filled by each of them in the literature. This makes it easier to understand what still needs to be done and what has room for enhancement.

Additional Key Words and Phrases: genome rearrangements, approximation algorithms, sorting permutations, comparative genomics

1 INTRODUCTION

Genome rearrangements are large-scale mutations affecting genomes. A classical problem in the Comparative Genomics field is estimating a minimal sequence of rearrangement events that may have happened between two genomes. This minimality comes from the commonly accepted parsimony principle. There are several challenges to make this estimation. It starts with which information will be retrieved from genomes, how they will be represented, and which genome rearrangements may be observed for that specific set of genomes.

These challenges led researchers to make choices that gave rise to sorting by rearrangement problems when using *permutations* to represent genomes, or to rearrangement distance problems when using other structures. Besides the biological motivation, these problems are relevant from a combinatorial point of view, as several of them have been proven to be NP-hard.

Authors' addresses: Andre Rodrigues Oliveira, andre.rodrigues@mackenzie.br, Faculdade de Computação e Informática, Universidade Presbiteriana Mackenzie, Rua da Consolação 896 - Consolação, São Paulo, SP, Brazil, 01302-907; Klairton Lima Brito, klairton@ic.unicamp.br; Alexsandro Oliveira Alexandrino, alexsandro@ic.unicamp.br; Gabriel Siqueira, gabriel.siqueira@ic.unicamp.br, Instituto de Computação, Universidade Estadual de Campinas (UNICAMP), Av. Albert Einstein 1251 - Cidade Universitária, Campinas, SP, Brazil, 13083-852; Ulisses Dias, ulisses@ft.unicamp.br, Faculdade de Tecnologia, Universidade Estadual de Campinas (UNICAMP), Rua Paschoal Marmo, 1888 - Jardim Nova Italia, Limeira, SP, Brazil, 13083-852; Zandoni Dias, zandoni@ic.unicamp.br, Instituto de Computação, Universidade Estadual de Campinas (UNICAMP), Av. Albert Einstein 1251 - Cidade Universitária, Campinas, SP, Brazil, 13083-852.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 0360-0300/2024/3-ART

<https://doi.org/10.1145/3653295>

When problems comparing genomes were proposed in the last century, most of the available data lacked information such as gene orientations. One of the first proposed genome representations was through unsigned permutations, where each number represents a particular gene of the genome. As sequencing evolved and became cheaper, not only the mathematical models incorporated gene orientation by adding a sign to each element in the permutation, but also other structures presented in the genome.

A good survey on problems in this field was presented in a book from 2009 by Fertin and coauthors [63]. As a lot of results have appeared after the book was published, we survey the advances of these approaches since 2010.

This survey complements others that have appeared recently. In 2018, Bhatia, Feijão, and Francis [21] provided an excellent introduction on how genomes can be modeled. Their survey describes different computational representations of genomes without replicated genes using permutations. In 2019, Zeira and Shamir [131] reviewed genome rearrangement problems with an emphasis on genomes with multiple chromosomes and copies of the same gene, and rearrangement events such as duplication and translocation, which, for this reason, will not be addressed in depth in our review. In 2019, Bulteau and Weller [37] presented an overview of parameterized algorithms for several bioinformatic problems, including also the comparison of genomes by rearrangement events. The focus of our survey is on complexity results and approximation algorithms instead.

2 PRELIMINARIES

In this section we give all the necessary definitions used in the following sections.

2.1 Genome Representations: Permutations, Strings and Intergenic Region Sizes

Given a pair of genomes being compared, there are several ways to represent them mathematically, and they vary mainly according to the assumptions of each particular research: if genomes have one or multiple chromosomes; if chromosomes are circular or linear; if the orientation of the genes inside chromosomes is known; if both genomes share the same set of genes or not; if repeated genes may happen or not; and finally, which structures from the genome will be used.

The first works in the field [15, 39, 44, 69, 79] represented genomes simply as lists of shared genes of genomes (expressions “genes” and “conserved regions/blocks” may be interchangeable here). Over time these representations evolved to allow genomes with unequal content and genomes with duplicated genes. Although the transformation between genomes can occur in both directions, we will refer to one of the genomes as the *source*, which will be transformed into the other genome called *target*.

In works assuming that genomes being compared (i) have one linear chromosome, (ii) share the same set of n genes, and (iii) have no duplicated gene, each genome is represented by a *linear permutation* (or simply permutation) $\pi^n = (\pi_1 \dots \pi_n)$, where π_i is an integer number representing the i -th gene, such that $1 \leq |\pi_i| \leq n$ and $|\pi_i| = |\pi_j|$ if, and only if, $i = j$. When gene orientations are known, a sign is used in each element to indicate the relative orientation: if genes have the same orientation on both genomes, the ‘+’ sign will be assigned to both corresponding elements in each representation; otherwise, we assign the ‘−’ sign to the corresponding element in the permutation representing the source genome and ‘+’ to the corresponding element in the permutation representing the target genome. A common variation of the representation of genomes by permutations, that is used to encode circular chromosomes, assumes that π_1 and π_n are either adjacent or the same element.

To avoid having to explicitly create a representation for both genomes, which would make the transformation process more complicated, we map the target genome as the *identity permutation*, represented as $\iota^n = (+1 \dots +n)$; if gene orientations of the genes are unknown, signs in ι^n are omitted. Then, we map the genes in the source genome as π^n , according to the assignment used in ι^n . By doing so we, in some sense, ignore one of the genomes: transforming π^n into ι^n reduces to a *sorting* problem, i.e., we just need to sort π^n .

Note that it is common for genomes being compared to have different sets of genes. One way to approach the comparison between them is to simply ignore genes present in only one genome and proceed with the previous representation based on permutations. However, more realistic models started to represent these unique genes in the permutation with a marker, just to inform the algorithms that a non-conservative event, i.e., a rearrangement that changes the content of a gene, must take place to transform one genome into the other. In this representation, models can use *insertions* and *deletions*, commonly abbreviated as *indels*, to add or remove the unique genes. Despite the advances in making more realistic models, these works still assumed the lack of repeated genes.

Some works [42, 111] removed the requirement of the uniqueness of genes, and in this case, instead of representing them as a permutation, they use *strings* such that repeated genes share the same representation in the string. This model is indeed more realistic, but it also makes it difficult to find solutions to the problems: most of the rearrangement problems that use strings have been proven to be NP-hard. A very common approach in these problems is to make an orthologous assignment between the genes in both strings, which allows these strings to be represented as permutations, thus being able to use the algorithms developed for permutations. Another natural generalization step in the genome representation is to adopt multiple strings [18, 130] to represent each genome, which allows the representation of multiple chromosomes.

Representing genomes through their gene order (as permutations or strings) causes loss of relevant information, including the genetic material not contained directly in the genes. In particular, the intergenic regions, that are DNA sequences between consecutive genes, are not considered by these representations. Recently, some authors argued that incorporating intergenic regions' sizes in the models improves the distance estimations [23, 24]. After that, researchers started considering a representation that can use both the gene order and intergenic region sizes. In these works, the gene order is also represented as permutation or string, but there is an additional list of intergenic region sizes, so instead of just transforming one sequence of genes into the other, these models also have to transform the list of intergenic region sizes from the source genome into the the list of intergenic region sizes of the target genome.

2.2 Rearrangements

In this section we define the large-scale rearrangements that are used in the works reviewed in this survey. Given a permutation π^n , we denote by $\pi^n \cdot \beta$ the application of a rearrangement β to π^n ; we also denote by $\pi^n \cdot S$ the application of the sequence of rearrangements S to π^n . The first works in the field investigated two of the most studied rearrangements, named *reversals* and *transpositions*, so let us start with them.

Given a permutation π^n , a *reversal*, denoted by $\rho(i, j)$, where $1 \leq i \leq j \leq n$, reverses the order and the signs of elements in positions i to j :

$$\begin{aligned}\pi^n &= (\pi_1 \dots \pi_{i-1} \quad \pi_i \dots \quad \pi_j \pi_{j+1} \dots \pi_n) \\ \pi^n \cdot \rho(i, j) &= (\pi_1 \dots \pi_{i-1} \quad \underline{-\pi_j \dots -\pi_i} \quad \pi_{j+1} \dots \pi_n)\end{aligned}$$

When reversals are applied to unsigned permutations it only reverses the order of the elements.

Given a permutation π^n , a *transposition*, denoted by $\tau(i, j, k)$, where $1 \leq i < j < k \leq n + 1$, moves the block of contiguous elements in positions from i to $j - 1$ to a new position between elements π_{k-1} and π_k :

$$\begin{aligned}\pi^n &= (\pi_1 \dots \pi_{i-1} \quad \pi_i \dots \pi_{j-1} \quad \pi_j \dots \pi_{k-1} \quad \pi_k \dots \pi_n) \\ \pi^n \cdot \tau(i, j, k) &= (\pi_1 \dots \pi_{i-1} \quad \underline{\pi_j \dots \pi_{k-1}} \quad \underline{\pi_i \dots \pi_{j-1}} \quad \pi_k \dots \pi_n)\end{aligned}$$

Observe that a transposition does not affect the sign of any element; so when dealing with signed permutations, we need to use transpositions together with other rearrangement events capable of changing gene orientations, such as reversals. It is worth noting that some works regarding genome rearrangement define the transposition

exchanging the sequence between i and j with the sequence between $j + 1$ and k instead of exchanging the sequence between i and $j - 1$ with the sequence between j and $k - 1$.

A generalization of transposition is the *block-interchange*, denoted by $BI(i, j, o, p)$, with $1 \leq i < j \leq o < p \leq n + 1$, that exchanges the position of blocks with contiguous elements in positions i to $j - 1$ and o to $p - 1$:

$$\begin{aligned}\pi^n &= (\pi_1 \dots \pi_{i-1} \pi_i \dots \pi_{j-1} \pi_j \dots \pi_{o-1} \pi_o \dots \pi_{p-1} \pi_p \dots \pi_n) \\ \pi^n \cdot BI(i, j, o, p) &= (\pi_1 \dots \pi_{i-1} \pi_o \dots \pi_{p-1} \pi_j \dots \pi_{o-1} \pi_i \dots \pi_{j-1} \pi_p \dots \pi_n)\end{aligned}$$

Note that a block-interchange $BI(i, j, j, p)$ has the same effect of a transposition.

There are also two operations derived from transpositions: one is a *transreversal*, denoted by $\tau\rho(i, j, k)$, that exchanges the blocks from i to $j - 1$ and j to $k - 1$ with an additional reversal of one of these two blocks (they are usually called type-1 or type-2 depending whether the reversed block is the first or the second one). Here is an example in which we reverse the block from j to $k - 1$:

$$\begin{aligned}\pi^n &= (\pi_1 \dots \pi_{i-1} \pi_i \dots \pi_{j-1} \pi_j \dots \pi_{k-1} \pi_k \dots \pi_n) \\ \pi^n \cdot \tau\rho(i, j, k) &= (\pi_1 \dots \pi_{i-1} \pi_k \dots \pi_{j-1} \pi_j \dots \pi_{k-1} \pi_i \dots \pi_n)\end{aligned}$$

The second rearrangement derived from transposition is a *revrev*, denoted by $\rho\rho(i, j, k)$, that reverses both blocks and the sign of all elements (if the permutation is signed), without exchanging the blocks:

$$\begin{aligned}\pi^n &= (\pi_1 \dots \pi_{i-1} \pi_i \dots \pi_{j-1} \pi_j \dots \pi_{k-1} \pi_k \dots \pi_n) \\ \pi^n \cdot \rho\rho(i, j, k) &= (\pi_1 \dots \pi_{i-1} -\pi_{j-1} \dots -\pi_i -\pi_{k-1} \dots -\pi_j \pi_k \dots \pi_n)\end{aligned}$$

All the above rearrangements are called *conservative* because they neither remove nor add new elements to the genome representation, they just rearrange them. However, when comparing genomes with different sets of genes we need to use *non-conservative* rearrangements to add the elements that are presented only in the target genome and to remove elements that are present only in the source genome. In this case, we can use an *insertion*, denoted by $\phi(i, T)$, to add the sequence T to the representation after the element at position i , and a *deletion*, denoted by $\psi(i, j)$, to remove elements between positions i and j from the representation; these two rearrangements together are called *indels*. Note that when representing genomes as permutations we stated that those genomes share the same set of unique genes; in the context of indels, the source genome is represented by a *string* σ^n ; if there are no repeated genes, the target genome may be represented by a permutation ι^m , and the string σ^n is labeled according to ι^m . See below an example of an insertion and a deletion.

$$\begin{aligned}\sigma^n &= (\sigma_1 \dots \sigma_i \sigma_{i+1} \dots \sigma_n) \\ \sigma^n \cdot \phi(i, (\sigma'_1 \dots \sigma'_m)) &= (\sigma_1 \dots \sigma_i \sigma'_{i+1} \dots \sigma'_{i+m} \sigma_{i+1} \dots \sigma_n) \\ \sigma^n &= (\sigma_1 \dots \sigma_{i-1} \sigma_i \dots \sigma_j \sigma_{j+1} \dots \sigma_n) \\ \sigma^n \cdot \psi(i, j) &= (\sigma_1 \dots \sigma_{i-1} \sigma_{j+1} \dots \sigma_n)\end{aligned}$$

Another well-studied rearrangement is the Double-Cut-And-Join (DCJ). As the name says, it cuts the genome representation at two adjacencies (a, b) and (c, d) and creates two new adjacencies by joining them: either (a, d) and (b, c) , or (a, c) and (b, d) . DCJs are generalizations of several rearrangements, including for instance a reversal. This rearrangement can be used in genomes with multiple circular or linear chromosomes, as long as we acknowledge that during the transformation of one genome into another we can have circular chromosomes.

In addition to DCJ, there is another rearrangement called *Multi-Cuts* or *k-Cuts*: it cuts k adjacencies and joins them back following some specific rules. This rearrangement also generalizes several rearrangements including reversal, transposition, block-interchange, and DCJ.

2.3 Models, distances and diameters

A *model* M states the set of rearrangements allowed to sort a permutation π^n .

Given a model M and a permutation π^n , the *distance*, denoted by $d_M(\pi^n)$, is defined as the *size*, i.e., the number of operations, in a minimum-sequence S such that $\pi^n \cdot S = \iota^n$ and all operations in S are allowed by M . We refer as *diameter*, denoted by $D_M(n)$, the maximum value of $d_M(\pi^n)$ when considering all permutations with n elements.

In this definition of distance, we can see that each rearrangement equally collaborates for the final value. However, in some populations in which more than one operation is observed, it may be the case that one particular operation is more common than others. One way to adjust the definition of distance to match this observation is by considering an operation-type weighted approach, such that each rearrangement has a predefined cost (so we expect more common operations to cost less). The *weighted distance* is defined as the cost of a minimum cost sequence S that sorts π^n , considering the cost of S as the sum of each operation's costs.

Note that rearrangements affecting long stretches may disturb the organisms in a more influential way, so the evolutionary pressure working against these rearrangements may be great, and in some organisms reversals were observed to be short [25]. For this situation some problems proposed a length-weighted approach, so an operation affecting ℓ elements has a cost that uses ℓ as a parameter. This approach, in some sense, penalizes long rearrangements. The *length-weighted distance* of a permutation π^n is defined as the cost of a minimum cost sequence S that sorts π^n , considering the cost of S as the sum of each operation's length-weighted costs.

Finally, there is also an approach in which the cost of an operation is defined as the number of cuts that this operation makes when applied in the permutation, called fragmentation-weighted approach. As the other weighted approaches, the *fragmentation-weighted distance* of a permutation π^n is then defined as the cost of a minimum cost sequence S that sorts π^n , considering the cost of S as the sum of each operation's fragmentation-weighted costs.

3 STRUCTURES AND METRICS

Several structures were proposed by researchers in order to guide the sorting process or to help devise bounds on the distances. In this section we briefly describe four structures that have been widely used after their proposals: breakpoints, strips, and the breakpoint and cycle graphs. These structures use an *extended* form of a permutation π^n , in which an element 0 is placed before π_1 and an element $(n+1)$ is placed after π_n ; these two extra elements help to check if the two extremities of the permutation have their correct elements. Besides, they are fixed, so no rearrangement affects them. It is worth noting that the same structures can also be used for circular genomes instead of extended linear genomes.

3.1 Breakpoints and Strips

Given an unsigned permutation π^n , we say that a pair of consecutive elements (π_i, π_{i+1}) , with $0 \leq i \leq n$, is a *breakpoint* if $|\pi_{i+1} - \pi_i| \neq 1$; otherwise, we say that this pair is an *adjacency*. On signed permutations we say that a pair of consecutive elements (π_i, π_{i+1}) is a *breakpoint* if $\pi_{i+1} - \pi_i \neq 1$. Those are the more common definitions of breakpoint, but they can be adapted depending on the model (e.g., if the model restricts operations to always be applied to π_1 , the pair (π_0, π_1) is always considered a breakpoint) or the representation of the genome.

We denote by $b(\pi^n)$ the total number of breakpoints in π^n . The identity permutation is the only permutation in which $b(\iota^n) = 0$, so when sorting a permutation, one can use the number of breakpoints as a guide and try to apply operations that decrease this number.

The maximum variation in the number of breakpoints after applying an operation β to a permutation π^n is related to the number of adjacencies affected by β . For instance, this variation is between -2 and +2 for reversals, and between -3 and 3 for transpositions. This observation usually results on lower bounds for sorting permutations.

Another concept widely used in the field and related to breakpoints is a *strip*, a maximal sequence of consecutive elements without breakpoints. See Fig. 1 for examples of breakpoints and strips.

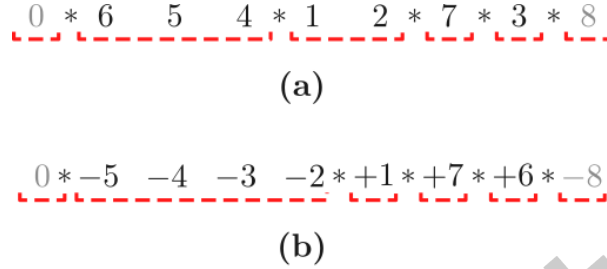


Fig. 1. In (a) we have the unsigned permutation $\pi = (6\ 5\ 4\ 1\ 2\ 7\ 3)$ in the extended form (the two extra elements are in gray) that has $b(\pi) = 5$ breakpoints (marked with asterisks); each strip is highlighted with dashed lines under the elements. In (b) we have the signed permutation $\pi' = (-5\ -4\ -3\ -2\ +1\ +7\ +6)$ also in the extended form and with $b(\pi') = 5$ breakpoints; each strip is highlighted with dashed lines under the elements.

Besides being used in problems involving other rearrangement distances, the number of breakpoints, called the *breakpoint distance*, is also used directly to measure the difference between two genomes.

3.2 Graph Structures

In addition to the breakpoint metric, two graph structures are widely used by works related to Sorting by Genome Rearrangements: the breakpoint graph and the cycle graph. Although they seem similar, they are used in different contexts: one is used for unsigned permutations, while the other is used on signed permutations or when the permutation is unsigned, but operations cannot reverse a sequence of elements (such as transpositions).

Given an unsigned permutation π^n , the *breakpoint graph* [14], denoted by $G_b(\pi^n)$, is defined as follows. The set of vertices is the elements of π^n in its extended form. There is a *reality edge*, also referred to as *black edge*, between two vertices π_i and π_{i+1} , for $0 \leq i \leq n$, if this pair is a breakpoint. Given two distinct vertices π_i and π_j , for $\{i, j\} \in [0..n+1]$, such that $|i - j| \neq 1$ (i.e., they are not adjacent), there is a *desire edge*, also referred to as *gray edge*, between them if this pair is an adjacency. Note that each vertex has up to two adjacent vertices and it has up to two adjacencies, so the maximum degree of any vertex in this graph is 4. See Fig. 2 for an example.

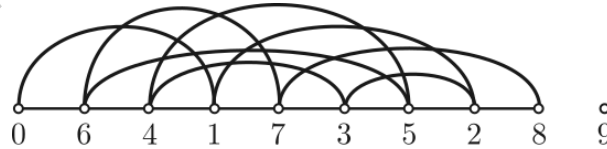


Fig. 2. The breakpoint graph $G(\pi)$ for the unsigned permutation $\pi = (6\ 4\ 1\ 7\ 3\ 5\ 2\ 8)$. The reality edges are represented in straight lines, while desire edges are represented by arcs. Note that, except for vertices 0, 8, and 9, any other vertex has degree 4. Besides, there is no reality nor desire edge between vertices 8 and 9, since they are not a breakpoint.

Given a signed permutation π^n , the *cycle graph* [15], denoted $G_c(\pi^n)$, is defined as follows. For each element π_i we add two vertices to $G(\pi)$, namely, $-\pi_i$ and $+\pi_i$. We also add two extra vertices: 0 and $-(n+1)$ (these are

the two elements in the extended permutation). We divide the set of edges into two sets: the *reality edges*, also referred as *black edges* are defined as $\{(-\pi_i, +\pi_{i-1}) : 1 \leq i \leq n+1\}$, and the *desire edges*, also referred as *gray edges* are defined as $\{+(i-1, -i) : 1 \leq i \leq n+1\}$. Note that reality edges link elements that are adjacent in π^n , while desire edges link elements that are adjacent in ι^n , which is our target.

There are multiple ways of drawing $G(\pi^n)$, but works in the literature tend to use the following convention: vertices are drawn from left to right, and for each element $\pi_i \in \pi^n$, with $1 \leq i \leq n$, the vertex $-\pi_i$ is drawn on the left of $+\pi_i$; 0 is the leftmost vertex in this line and $-(n+1)$ is the rightmost vertex. Vertices of element π_i are placed immediately before vertices of π_{i+1} . The black edges are drawn as straight lines, while the gray edges are drawn as arcs above the vertices.

In the cycle graph, each vertex is incident on one black edge and one gray edge, so a cycle graph is a collection of components so that each connected component is a cycle, i.e., $G_c(\pi^n)$ allows a unique decomposition of edges in cycles. We note that each cycle has the same number of black and gray edges, since there are no two consecutive edges of the same type when traversing the cycle. As already mentioned, the goal of a generic Sorting by Genome Rearrangements problem is to transform a permutation π^n into ι^n . We denote by $c(G(\pi^n))$ the number of cycles in $G(\pi^n)$. We note that $G(\iota^n)$ is the only cycle graph where $c(G(\iota^n)) = n+1$, which is the maximum number of cycles by construction, so in order to sort π^n one needs to increase the number of cycles in $G(\pi^n)$ to $n+1$.

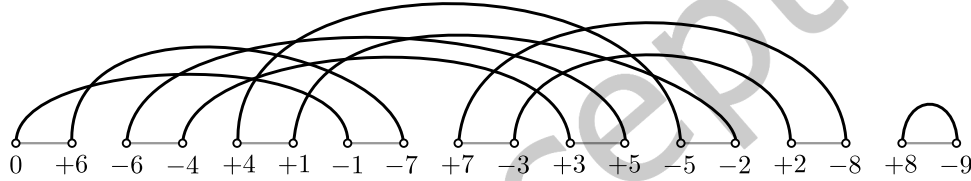


Fig. 3. The cycle graph $G(\pi)$ for the signed permutation $\pi = (-6 +4 -1 +7 +3 -5 +2 +8)$. Note that every vertex has degree 2 and there are 5 cycles, being 4 with 2 black edges and 1 with one black edge.

Note that the breakpoint graph has multiple gray edges incident in each vertex, hence it does not have a unique decomposition into cycles as is the case with the cycle graph. To remediate that fact, an unsigned permutation π^n can be mapped as a signed permutation π_s^n , by assigning arbitrary signs ('+' or '-') to each element of π^n . After that, $G(\pi_s^n)$ can be associated to π^n . For instance, the Sorting by Transpositions problem, that uses unsigned permutations since transposition do not change the orientation of the elements, uses the cycle graph structure by assigning a '+' sign to every element of π_n : this guarantees that π_s^n can be sorted to the signed version of ι^n . However, for other operations that modify the signs of elements this is only one of the possible cycle graphs for π^n . This means that π^n can be mapped to several different cycle graphs (of distinct signed permutations generated from π^n). Since the goal is to increase the number of cycles to $n+1$, the best graph here is the one that maximizes the number of cycles. However, finding a decomposition that maximizes the number of cycles in the cycle graph is an NP-hard problem [39]. Consequently, works on the area use approximation algorithms to find a reasonable cycle graph when they want to use the cycle graph from an unsigned permutation. At the moment, the best approximation algorithm for the cycle decomposition has approximation factor of $1.4167 + \epsilon$, for $\epsilon > 0$, proposed by Jiang and coauthors in 2014 [76].

Note that maximizing the number of cycles in the cycle graph by attributing signs for each element of an unsigned permutation can also be seen as finding the maximum number of disjoint alternated cycles in the breakpoint graph. This problem can be furthered generalized to any indirect acyclic graph with some edges marked as black and others as gray [22].

When working with the cycle graph, it is important to observe how each operation affects the graph, i.e., how many cycles can be generated if we apply a particular operation. For instance, a reversal can modify $c(G(\pi^n))$ by

1, 0, or -1, while a transposition can modify $c(G(\pi^n))$ by 2, 0, or -2. Different classifications of cycles may also be considered, such as *odd* (resp. *even*) for cycles with an odd (resp. even) number of edges of black edges, and so on. For instance, in Fig. 3 the rightmost cycle is odd, while all other cycles are even.

It is worth mentioning that there is also the adjacency graph [18], which is better suited to represent multiple chromosomes. For more details on graphic representation allowing multiple chromosomes see the section on genome representation in the survey from Zeira and Shamir [131].

3.3 Simple Permutations

Some of the most important results on Sorting by Genome Rearrangements use the concept of *simple permutations* [57, 69]. Indeed, working with this type of permutation helps limit how the cycle graph looks. Let us briefly describe this class of permutations and how to transform any permutation into a simple permutation.

Given a permutation π^n , a cycle $C \in G(\pi^n)$ is said to be *short* if it has 3 or fewer black edges; otherwise C is called *long*. A permutation π^n is called *simple* if its cycle graph $G(\pi^n)$ has only short cycles.

If a permutation π^n is not simple, it is possible to transform π^n into a simple permutation $\hat{\pi}^n$ by adding new elements to π^n that break long cycles [69]. This procedure is called *safe* if both π^n and $\hat{\pi}^n$ have the same lower bound.

A sequence that sorts $\hat{\pi}^n$ can be adapted to sort π^n , with the same number of operations or less, simply by ignoring the elements added to $\hat{\pi}^n$; operations affecting only added elements can be completely ignored.

4 SORTING BY CLASSICAL OPERATIONS

In this section we review the classical problems and list the best achievements. We divide the results by the set of operations used in each problem. Table 1 highlights the main results and indicates the models that can affect multiple chromosomes.

Table 1. Summary of the main results for classical operations. The column MC indicates if the model can be applied to compare genomes with multiple chromosomes.

Orientation	Model	Complexity	Best Approximation	MC
Unsigned	Reversals	NP-hard [39]	1.375 [20]	No
Unsigned	Transpositions	NP-hard [34]	1.375 [57]	No
Unsigned	Reversals and Transpositions	NP-hard [98]	$2k$ [110]	No
Unsigned	Block interchanges	P [44]	-	Yes
Unsigned	Double-Cut-and-Joins	NP-hard [40]	$\frac{17}{12}$ [41]	Yes
Unsigned	k -Cut ($k > 4$)	NP-hard [32]	2 [32]	Yes
Signed	Reversals	P [69]	-	No
Signed	Reversals and Transpositions	NP-hard [98]	2 [127]	No
Signed	Double-Cut-and-Joins	P [18, 130]	-	Yes
Signed	Single-Cut-or-Joins	P [59]	-	Yes
Signed	k -Cut ($k > 4$)	NP-hard [97]	$(\frac{3k}{2k-8} + \epsilon)$ [97]	Yes

4.1 Reversals

In 1995, Kececioglu and Sankoff [79] presented the first algorithm for Sorting by Reversals (SbR) on unsigned permutations, using breakpoints and strips, with an approximation factor of 2. Algorithms based on the decomposition of the breakpoint graph of an unsigned permutation π^n improved this approximation factor to 1.75 in 1996 [14], to 1.5 in 1998 [45], and the best approximation factor is 1.375 [20], proposed by Berman and coauthors in 2002. Caprara proved that SbR is NP-hard on unsigned permutations in 1999 [39], by showing a strong relation between the problem of maximum cycle decomposition of the breakpoint graph and the reversal distance.

For signed permutations, however, Hannenhalli and Pevzner showed in 1999 that it is possible to find the reversal distance in polynomial-time [69]. After this discovery, researchers focused on simplifying the original algorithm or improving the time complexity. In 2005, this algorithm was simplified by Bergeron [17], and there is a subquadratic time algorithm for computing the sorting sequence [125] and also a linear time algorithm if only the distance is needed [12].

A variant of SbR is the Sorting by Length-Weighted Reversals (SbLWR), where we are interested in the length-weighted distance. Pinter and Skiena [108] studied this variant on unsigned permutations in 2002 and presented an approximation algorithm with factor $O(\log^2 n)$. This factor was improved by Bender and coauthors in 2008 to $O(\log n)$ [16]. This variant was also studied on circular and signed permutations by Swidan and coauthors, that devised approximation algorithms with factor $O(\log n)$ [124].

In 2018, Arruda and coauthors [49] also investigated the SbLWR on signed and unsigned permutations and devised a new algorithm based on the Greedy Randomized Adaptive Search Procedure (GRASP) metaheuristic that performed better than other algorithms in simulated tests. The computational complexity of the length-weighted variant is unknown.

El-Mabrouk [56] studied the Genome Rearrangement Distance by Reversals and Indels (GRDRI) when considering genomes with gene orientations and unequal set of genes but with no duplicated genes. They presented a polynomial-time algorithm when only insertions (or only deletions) are allowed and a heuristic when both non-conservative events are allowed. In 2013, Willing and coauthors showed a class of permutations that also admits a polynomial-time algorithm when both non-conservative events are allowed [129]; this result was generalized in 2021 [128] when the authors showed that GRDRI problem is polynomial-time solvable. When gene orientation is not considered, GRDRI is NP-hard, and there is a 2-approximation algorithm based on breakpoints [8].

4.2 Transpositions

Bafna and Pevzner presented in 1998 the first approximation algorithm for Sorting by Transpositions (SbT), with an approximation factor of 1.5 [15], based on the cycle graph. In 2006, Elias and Hartmann [57] improved the approximation factor to 1.375, the best factor so far. In 2012, Bulteau and coauthors [34] showed that SbT is NP-hard, using a polynomial-time reduction from SAT. These authors also showed that even deciding whether it is possible or not to sort a permutation π^n by exactly $\frac{b(\pi^n)}{3}$ transpositions is NP-hard.

In 2013, Dias and Dias [51] presented heuristics for SbT that also guarantee the 1.375 approximation factor; according to practical experiments, these heuristics outperform other algorithms for SbT on small permutations and keep good performance on longer permutations. In 2015, Cunha and coauthors [48] presented an approximation algorithm based on Elias and Hartmann with a faster running time.

Silva and coauthors found a problem in the Elias and Hartmann (EH) algorithm [114]. The EH algorithm uses the simplification process to work only with simple permutations. Silva and coauthors showed that this simplification step, that is the first step of EH, can make the algorithm apply one transposition above the approximation factor. The authors corrected the algorithm using an algebraic approach that runs in $O(n^6)$. In 2022, Alexandrino and coauthors [3] presented an alternative correction using permutations and the cycle graph that runs in $O(n^5)$.

The length-weighted approach of SbT, denoted by SbLWT, was investigated in 2008 by Qi and coauthors [109]. Using the cost function $f = \ell^\alpha$, where ℓ is the number of affected elements, they presented an $O(\lg n)$ -approximation algorithm for $\alpha < 2$ and showed that the problem becomes polynomial-time solvable when $\alpha \geq 2$.

In 2020, Alexandrino and coauthors [8] showed that Genome Rearrangement Distance by Transpositions and Indels (GRDTI) is NP-hard; in 2022 the same authors presented a 2-approximation algorithm for GRDTI using a graph structure based on the cycle graph [10].

4.3 Reversals and Transpositions

The first algorithm for Sorting by Reversals and Transpositions (SbRT) was presented in 1998 by Walter and coauthors [127]. For signed permutations, they presented a 2-approximation algorithm using the cycle graph structure. In 1999, Gu and coauthors [68] introduced the transreversal rearrangement. By allowing this third rearrangement operation in the model the authors showed a 2-approximation algorithm for signed permutations. Lin and Xue [87] also introduced a new rearrangement called revrev, and by combining the four rearrangements they showed a 1.75-approximation algorithm. Hartman and Sharan [70] improved the 1.75 factor to 1.5 by using simple permutations and the cycle graph. These problems were shown to be NP-hard in 2020 by Alexandrino and coauthors [7].

For unsigned permutations Walter and coauthors [127] presented a 3-approximation algorithm based on breakpoints. Rahman and coauthors [110] improved this factor to $2k$ where k is the approximation factor of the algorithm used for the cycle decomposition of a breakpoint graph (at the moment the best factor is $1.4167 + \epsilon$ [76]). Oliveira and coauthors [100] presented heuristics for SbRT, and showed that they outperformed the other algorithms for SbRT in the literature.

In 2019, Oliveira and coauthors [98] showed that Sorting by Reversals and Transpositions is NP-hard on signed and unsigned permutations.

All the algorithms mentioned so far in this section assume that operations have the same cost. However, since a transposition fragments more a permutation than a reversal, it can remove more breakpoints and create more cycles, so, commonly, those algorithms produce solutions biased toward transpositions. One way to overcome this issue is the use of weighted and fragmented distances. Another motivation for the weighted approach is when a specific rearrangement is more likely to occur than others in a particular population [25, 130].

In 2002, Eriksen [58] presented a $\frac{7}{6}$ -approximation algorithm for Sorting by Weighted Reversals, Transpositions and Transreversals (SbWRTT), where the weight of a reversal is 1, and the weight of a transposition or a transreversal is 2. In 2007, Bader and Ohlebusch [13] developed a 1.5-approximation algorithm for SbWRTT, where the weight of a reversal is 1 and the weights of the other rearrangements are in the range $[1..2]$.

Oliveira and coauthors [99] presented a 1.5-approximation algorithm for Sorting by Weighted Reversals and Transpositions where the weight of a reversal is 2 and the weight of a transposition is 3. The authors also investigate when the ratio between the cost of a transposition and the cost of a reversal is in the interval from 1 to 3, and showed that in these cases the approximation factor is no greater than 2. Oliveira and coauthors [98] also showed that Sorting by Weighted Reversals and Transpositions is NP-hard when the ratio between the weight of a transposition and the weight of a reversal is less than or equal to 1.5 on both signed and unsigned permutations.

Alexandrino and coauthors [5] studied the fragmentation-weighted approach with reversals and transpositions; it is similar to the weighted approach when the weight of a reversal is 2 and the weight of a transposition is 3, but when these operations are applied to the extremities of the permutation, the weights are lower. They presented 2-approximation algorithms for signed and unsigned permutations considering reversals and transpositions, and a 1.5-approximation algorithm for Sorting by Fragmentation-Weighted Transpositions on simple (unsigned) permutations.

4.4 Generalized Rearrangements

In this section, we briefly comment results of more generic operations: Double-Cut-and-Join (DCJ), Single-Cut-or-Join (SCJ), block-interchange (BI), and multi-cuts. One important aspect of those operations is that they can affect multiple chromosomes. More information in that scenario with multiple chromosomes can be found on the survey from Zeira and Shamir [131].

DCJ is a generalization of several rearrangements such as inversions, translocations, fusions, and fissions. This operation cuts the genome into two adjacencies and combines the resulting pieces. It was proposed in 2005 by Yancopoulos and coauthors [130]; Sorting by DCJs is polynomial-time solvable on signed permutations [130] and there are linear-time algorithms for computing the edit distance under DCJs [18, 19]. The problem is NP-hard on unsigned permutations [40]. Several algorithms using DCJs have been proposed taking advantage of the fact that it has an exact algorithm on signed permutations [26, 61, 62].

The DCJ operation generalizes many rearrangement operations and was well studied in the classical setting, so it may be considered a good starting point for exploring rearrangement distance problems with additional constraints. For example, Swenson and coauthors [122] considered a weight associated to each DCJ operation based on the likelihood of cutting each pair of adjacencies. They proposed an algorithm to find a sequence of DCJ with minimal cost between the sequences of minimal length that sort the permutation. Simonaitis and Swenson also studied this problem when considering the number of operations (distance) instead of the length-weighted approach [123]. They showed that the problem is NP-hard and presented a 1.5-approximation algorithm for it. This approach is particularly useful when considering the effect of physical proximity of the genes on the likelihood of each operation. Later, Simonaitis and coauthors [115] generalized this approach by proposing a framework that uses arbitrary functions to attribute the weights for the operations. Their framework expands a solution for a single cycle of the breakpoint graph into a solution for the whole instance, simplifying the creation of exact and approximation algorithms. As in the previous case, this generalization can be solved in polynomial time if the sequence of DCJs is required to have minimal length [116]. It is worth mentioning that DCJ is widely adopted when comparing genomes with multiple chromosomes and was also explored in combination with gene duplication. The survey from Zeira and Shamir [131] presents a complete discussion about those studies with DCJ. An interesting new approach to compare genomes with DCJ is the problem of unifying adjacencies by DCJs proposed by Sun and Jiang [121]. In this problem, instead of transforming the source genome into the target genome, the goal is to transform the source genome into a new genome with the same adjacencies as the target genome. The authors proved that this problem is NP-hard when genes can have multiple copies and proposed a $(\frac{13}{9} + \epsilon)$ -approximation algorithm for the problem.

Inspired by the DCJ operation, Feijão and Meidanis [59] proposed a simpler rearrangement distance called Single-Cut-or-Join (SCJ), which cuts a genome segment into two segments or combines two genome segments. The authors explained how the operation generalizes the breakpoint distance to multiple chromosomes and proposed a polynomial time algorithm to find it. As a consequence of its simplicity, the SCJ distance was also used in the exploration of more complex problems, such as the distance and median problem with SCJ and gene duplication [91] and the problem of counting or sampling optimal solutions [93].

A block-interchange generalizes transpositions, since the exchanged blocks do not need to be adjacent. In 1995, Christie [44] showed a polynomial-time algorithm for Sorting by Block-Interchanges (SbBI) on unsigned permutations. In 2009, Lin and coauthors [88] investigated a weighted approach using block-interchanges and reversals, and presented approximation algorithms when the weight of a reversal is 1 and the weight of a block-interchange is between 2 and 3.

Finally, a Multi-Cut or a k -Cut operation is a generalization of several rearrangements, including reversals, transpositions, block-interchanges and DCJs. It was introduced by Alekseyev and Pevzner in 2008 [1, 2]. This new rearrangement may model *chromoanagenesis* and *chromotripsis*, phenomena consisting of massive simultaneous

rearrangements [74, 106, 120]. On unsigned permutations, Bulteau and coauthors showed in 2021 that sorting by k -Cut is NP-hard for $k = 3$ (since in this case a k -cut is a transposition) and for $k \geq 5$ [32]; in this work the authors do not consider that blocks can be reversed. Oliveira and coauthors investigated the Sorting by k -Cuts on signed permutations, allowing block reversals. They proved that the problem is NP-hard for $k \geq 5$, and showed a 1.5-approximation algorithm for $k = 4$ and a generic $(\frac{3k}{2k-8} + \epsilon)$ -approximation algorithm for any $k \geq 5$. We note however that the complexity of Sorting by k -Cuts on signed permutations remains open for $k = 4$. In the context of multiple chromosomes, Simonaitis and Raphael [117] propose the problem of finding a sequence of *multi-breaks* (a k -Cut with any k) that minimizes the number of chromosomes broken by any operation. They propose an exact algorithm for the problem, assuming that no position can be broken more than once. We note that their algorithm is not polynomial and the complexity of this problem still remains open.

5 SORTING BY POSITION-RESTRICTED OPERATIONS

Given a permutation or a string, prefix and suffix operations are those whose first and last elements are always among the affected elements, respectively. For instance, given a permutation π^n , a prefix reversal is always represented by $\rho(1, j)$ and a suffix reversal is represented by $\rho(i, n)$, for some $\{i, j\} \in [1..n]$.

The prefix operations are less applicable in genome rearrangement problems, but they can also be used in the interconnection network problem. A processor in this case can be modeled as a permutation, and there is a link between two processors if one can be transformed into the other using a particular operation. The maximum communication delay, defined as *diameter*, can be reduced when we restrict to prefix operations, since those operations reduce the size of the generated networks [85].

Another closely-related problem that uses a prefix operation is the Pancake Flipping problem [55] (PFP), which can also be seen as the Sorting by Prefix Reversals (SbPR). In this problem, given a stack of pancakes of distinct diameters, the goal is to find the minimum number of movements to rearrange the pancakes from the smallest on the top to the highest on the bottom. The only allowed movement is to use a spatula placed under one or more pancakes and flip all pancakes above it. It is important to clarify that, unlike sorting problems, PFP is dedicated to finding the diameter of a stack with n pancakes, and not distances itself.

Similar to the signed and unsigned versions for permutations, the Pancake Flipping problem has the *burnt* variant, introduced by Gates and Papadimitriou [67], in which every pancake has two sides: a burnt one and an unburnt one (in the original problem we can consider both sides unburnt), and in addition to rearrange pancakes, it requires that all pancakes have its burnt side down. This problem was also generalized for k -sided pancakes [38, 78], so each pancake can have multiple sides of different colors for example, but only pancake networks (i.e., graphs and diameter aspects) were investigated.

In the following sections, we overview several works and recent advances in different sets of operations. Before starting, we highlight that although these models restricted to prefix or prefix and suffix operations have been extensively investigated in recent years, most have unknown computational complexity so far.

5.1 Prefix Reversals

As previously said, this problem was first investigated as the PFP and network problems, but the focus was on finding the diameter. This problem was introduced in the context of sorting permutations problem and distance in 1995 [107]. The best approximation algorithm for SbPR on unsigned permutations has an approximation factor of 2, proposed in 2005 by Fischer and Ginzinger [64], and it was built based on the cycle graph.

One of the main advances in the last years is a proof that this problem is NP-hard in 2015 proposed by Bulteau and coauthors [35]; the authors used a reduction from the 3-SAT problem to show that even deciding if a permutation π can be sorted using exactly $b(\pi)$ prefix-reversals is NP-hard. Another advance, introduced by

Cibulka in 2011 [46], is a formula for the diameter of the problem ($\lfloor \frac{3n+3}{2} \rfloor \leq D(n) \leq 2n - 6$), and the exact values for $d(\pi^n)$ for $n \leq 19$.

The corresponding variant of SbPR on signed permutations has a 2-approximation algorithm, presented in 2005 [47]. One advance in the past years that is worth mentioning is the characterization of a class of permutations that can be sorted in polynomial-time, proposed by Labarre and Cibulka [83]. In this work, the authors call a permutation *simple* if its cycle graph has no cycle of size greater than 2; by showing a new lower bound for the problem, the authors also show an exact formula for computing the distance on simple permutations. Similar to SbPR on unsigned permutations, Cibulka showed in 2011 [46] the exact values for $d(\pi^n)$ for $n \leq 17$. To this date SbPR on signed permutations remains with unknown complexity.

5.2 Prefix Transpositions

In 2002, Dias and Meidanis [54] introduced the Sorting by Prefix Transpositions (SbPT) and showed a 2-approximation algorithm based on breakpoints and strips; this is the best approximation factor in the literature. They also presented bounds for the diameter, but the lower bound was improved to $\lfloor \frac{3n}{4} \rfloor$ using an algebraic reinterpretation of the cycle graph by Labarre in 2013 [81], and the later was improved to $n + \log_{7/2} n$ by Chitturi in 2015 [43]. The complexity of SbPT remains unknown.

5.3 Prefix Reversals, Transpositions and Transreversals

The Pancake Flipping with Two Spatulas problem was introduced in 2010 by Hasan and coauthors [71]; by up to two spatulas we can simulate the following rearrangements: prefix reversals, prefix transpositions and prefix transreversals. Using a graph structure the authors showed a 3-approximation algorithm for Sorting by Prefix Reversals and Prefix Transpositions (SbPRT) and a 2-approximation algorithm for Sorting by Prefix Reversals, Prefix Transpositions and Prefix Transreversals (SbPRTT) on unsigned permutations. In 2015, Dias and Dias [53] presented an asymptotic 2-approximation algorithm based on strips for SbPRT on unsigned permutations, the best approximation factor in the literature. The complexity of SbPRT and SbPRTT are both unknown.

5.4 Prefix and Suffix Reversals and Transpositions

This problem was proposed in 2014, combining both reversals and transpositions restricted not only to prefixes but also suffixes [89, 90]. Lintzmayer and coauthors showed in a database of simulated genomes that, by adding the suffix operations in problems SbPR, SbPT, and SbPRT on signed and unsigned permutations, the distance can decrease by more than 60% on average when transpositions are not allowed, and by up to 50% on average otherwise [90].

Let SbPSR, SbPST, and SbPSRT be the corresponding problems that allow suffix operations on problems SbPR, SbPT, and SbPRT, respectively. The authors [90] also presented 2-approximation algorithms for SbPSR on signed and unsigned permutations and for SbPST problems, and an asymptotic 2-approximation algorithm for the others. The authors also show the following bounds for the diameter on unsigned permutations: between $n - 1$ and n for SbPSR, between $\lceil \frac{n-1}{2} \rceil + 1$ and $n - \lfloor \frac{n}{4} \rfloor$ for SbPST, and between $\lceil \frac{n-1}{2} \rceil$ and $\lceil \frac{n}{2} \rceil + 1$ for SbPSRT; and the following bounds for the diameter on signed permutations: between n and $n + \lfloor \frac{n-1}{2} \rfloor$ for SbPSR and between $\lceil \frac{n}{2} \rceil + 1$ and $n + 2 - \lfloor \frac{n}{4} \rfloor$ for SbPSRT.

In another work, Lintzmayer and coauthors [89] investigated the use of models with different sets of prefix and suffix operations applied to permutations and binary strings, i.e., strings containing only 0s and 1s. Instead of assuming that each operation costs one to the distance, they used the length-weighted approach. They presented results considering two cost functions: one is $f(\ell) = \ell^\alpha$ for some $\alpha > 0$ and the other is $f'(\ell) = 2^\ell$, where ℓ is the length defined as the number of elements affected by the operation.

Among the problems mentioned in this section, only one of them has known complexity: in 2018, Fertin and coauthors [60] showed that SbPSR on unsigned permutations is NP-hard using a reduction from the strict version of SbPR, where we want to know if it is possible to sort a permutation π^n using exactly k prefix reversals, where k is the number of prefix breakpoints in π^n .

5.5 Prefix Block-Interchange

The Prefix Block-Interchange (SbPBI) was introduced by Labarre in the context of (unsigned) permutations in 2020 [82]. The author used the cycle graph structure to show bounds for the distance and presented a 2-approximation algorithm for the problem. He also proved an upper bound for the diameter of SbPBI: $\lfloor \frac{2n}{3} \rfloor$. In 2021, Pai and Chitturi [105] presented a simpler 2-approximation algorithm for SbPBI (using the name of extreme block-interchange instead prefix block-interchange). They also showed an exact algorithm for sorting the permutations of the form $\pi^n = (n \ n-1 \ \dots \ 2 \ 1)$, and a $\frac{4}{3}$ -approximation algorithm for a family of permutations with a particular type of cycle graph. The complexity of the problem is unknown.

5.6 Prefix Double Cut and Join

In 2022, Fertin and coauthors [61] introduced the Sorting by Prefix Double Cut and Join (SbPDCJ). They presented lower bounds for the distance using the breakpoint graph structure, both on signed and unsigned permutations. As we explained in Section 3.2, the breakpoint graph of an unsigned permutation admits more than one decomposition in cycles, but in this prefix version the authors seek for a decomposition that maximizes the number of trivial cycles and minimizes the number of non-trivial cycles, and they showed that this task can be performed in polynomial-time. The authors also showed that the lower bound for SbPDCJ on signed permutation is tight, and that it is always possible to apply a prefix DCJ that decreases this lower bound by one, so it results in a polynomial-time algorithm for SbPDCJ. For unsigned permutations, they showed a 1.5-approximation algorithm for SbPDCJ based on breakpoints, and also a FPT algorithm parameterized with respect to the number of breakpoints. The SbPDCJ has unknown complexity on unsigned permutations; besides, it is worth mentioning that the algorithm presented by the authors for this problem is the only approximation algorithm for prefix operations with an approximation factor lower than 2.

6 SORTING BY SIZE-RESTRICTED OPERATIONS

Another variant proposed in the literature considers a restriction on the number of elements affected by each operation. A λ -operation is an operation affecting λ elements of the permutation or string. An operation is said to be *short* if $\lambda \leq 3$ and *super short* if when $\lambda \leq 2$. These models are motivated in the comparative genomics field by the assumption that rearrangements affecting a large stretch of the genome are less likely to occur, and also by the evidence of small rearrangements in some populations [50, 86, 112].

We recall that size-restricted operations are different from the length-weighted approach. While the former allows only operations affecting up to a fixed number of elements, the latter does not restrict the number of elements affected by an operation, and it penalizes operations affecting more elements at a higher cost than operations affecting fewer elements.

Many algorithms dealing with short operations use the concept of an *inversion*, that is a pair of elements $(\pi_i, \pi_j) \in \pi^n$ such that $i < j$ and $|\pi_i| > |\pi_j|$. Despite the first problems involving short operations appearing in the 1980s, many of the advances in the area have occurred in the last ten years. We note that all problems mentioned in the remaining of this section that use short operations have unknown complexity to this date. Recall that, as mentioned in Section 2.1, a permutation can be linear or circular. In case of a circular permutation, genome rearrangements may affect elements that are in the extremities without modifying the elements in the middle; some results for circular permutations are known for super short operations, that, in this case, were defined as

cyclic genome rearrangements. Cyclic genome rearrangements are particularly useful in the comparison between bacterial genomes. See Table 2 for a summary of the results.

Table 2. Summary of results for short and super short operations for linear and circular permutations.

Permutation Type	Model	Complexity	Best Approximation
Unsigned	Short Reversals	Unknown	5 [65]
Unsigned	Short Transpositions	Unknown	$\frac{5}{4}$ [76]
Unsigned	Short Reversals and Transpositions	Unknown	3 [65]
Unsigned	Super Short Operations	P [80]	-
Unsigned	Cyclic Super Short Operations	P [75]	-
Signed	Super Short Reversals	P [65]	-
Signed	Super Short Operations	P [65]	-
Signed	Cyclic Super Short Reversals	P [66]	-
Signed	Cyclic Super Short Operations	P [101]	-

6.1 Sorting Unsigned Linear Permutations by Short Operations

When dealing with unsigned linear permutations, any conservative operation must affect at least two elements; otherwise, the operation has no effect when applied. For this reason, a super short operation must affect exactly two elements. The same is not valid for signed permutations: a super short reversal may affect only one element and change its sign. Also, transpositions exchange two adjacent blocks, so each block has at least one element. For this reason, no transposition affecting only one element of the permutation is possible by definition.

Jerrum [75] presented in 1985 a polynomial-time algorithm for the Sorting by Swaps problem (SbSw), which is exactly the number of inversions in π^n , that also matches the *bubble sort* algorithm presented by Knuth [80, p. 108]. A swap in this case can be a super short reversal or a super short transposition. In 2003, Heath and Vergara [73] studied the Sorting by Short Reversals (SbShR), and by using a structure called *diagram vector*, that captures how many swaps and at what direction an element needs to go to its correct position, they devised a 2-approximation algorithm. They also showed that deciding if a permutation can be sorted optimally (i.e., using the number of short reversals equal to the lower bound based on the diagram vector) in quadratic time is possible; they called such permutations as *lucky*.

In 2021, Zhang and coauthors investigated a problem called “Sorting by Best Short Swaps” [132]; in this case a short swap can be either a short reversal, or a short transposition. They showed that it is possible to decide if a permutation can be sorted optimally (i.e., using a number of short operations equal to the lower bound based on number inversions and diagram vectors) in linear time; those permutations are called *lucky* for the lower bound based on diagram vectors, as previously defined by Heath and Vergara [73], and *happy* for the lower bound based on the number of inversions.

Heath and Vergara also investigated the Sorting by Short Transpositions problem (SbShT) and, by using a structure called *permutation graph*, that shows how inversions are connected, they devised a $\frac{4}{3}$ -approximation algorithm for SbShT in 2000 [72]; this approximation factor was improved to $\frac{5}{4}$ in 2014 by Jiang and coauthors [76].

In 2012, Jiang and coauthors [77] devised an $(1 + \epsilon)$ -approximation algorithm for SbShT, where ϵ is a relation between the number of elements and the number of inversions in π^n , so it is applicable on permutations with a

high number of inversions. In 1998, Vergara showed a 2-approximation algorithm for Sorting by Short Reversals and Transpositions problem (SbShRT) [126].

6.2 Sorting Signed Linear Permutations by Short Operations

In 2015, Galvão and coauthors [65] introduced problems with short operations to signed permutations. They used two structures: the diagram vector and the permutation graph. By using both, and analyzing the number of negative elements in each connected component in the graph, they showed that on signed permutations the Sorting by Super Short Reversals (SbSShR) and the Sorting by Super Short Reversals and Transpositions (SbSShRT) problems admit a polynomial-time algorithm. They also showed for signed permutations a 5-approximation algorithm for SbShR and a 3-approximation algorithm for SbShRT.

6.3 Sorting by Super Short Operations on Circular Permutations

So far, we have presented works dealing with linear permutations. Now we present results for circular permutations in both signed and unsigned versions. Note that, to solve a sorting problem that uses a circular representation, we can instead use the result from the corresponding problem (i.e., same set of operations) on linear permutations but considering that every operation is *cyclic*, i.e., it can affect a sequence composed by a suffix followed by a prefix of the permutation. By solving the latter, we just need to transform the (original) circular permutation π^n into the $2n$ possible linear permutations π'^n , run the algorithm with circular operations on linear permutations, and return the lowest distance among them. The results for circular permutations use this fact and investigate circular operations on linear permutations instead. Only circular super short operations were studied, and we list the results below.

Jerrum [75] presented in 1985 a polynomial-time algorithm for the Sorting by Cyclic Swaps problem (SbCSw) on unsigned permutations; the proof uses a vector structure called *displacement vector*, that captures the number of *cyclic swaps* (cyclic super short reversal or cyclic super short transposition) needed to sort the permutation.

In 2017, Galvão and coauthors [66] presented a polynomial-time algorithm for the Sorting by Cyclic Super Short Reversals (SbCSShR) on signed permutations; their algorithm also uses the displacement vector structure to capture the number of *cyclic reversals* needed, with an extra function that counts the number of positive (resp. negative) elements involved in an odd (resp. even) number of swaps; in these cases, the elements will be in their correct positions at the end, but with negative signs, so each of them will require an extra reversal to be positive again.

In 2018, Oliveira and coauthors [101] presented a polynomial-time algorithm for the Sorting by Cyclic Super Short Reversals and Transpositions (SbCSShRT) on signed permutations; as the other two works in this section it uses the displacement vector and also a graph structure to check negative elements in components of needed swaps; this is necessary since in this case both a super short reversal and a transposition swap two adjacent elements (or the first one with the last one, since these operations are cyclic), but the reversal also change the signs of both elements while a transposition keeps the same signs, so the components will help at each step to choose between a reversal or a transposition.

6.4 Sorting Permutations by Generic λ -Operations

In addition to short and super short operations, there are works that focused on λ -operations for any value of λ .

In 2005, Nguyen and coauthors [96] investigated the Sorting by λ -Reversals using the length-weighted distance such that the cost is exactly the length of the operation. They showed a $(\lg n)$ -approximation algorithm when $\lambda = \Omega(n)$ and a $(2 \lg^2 n + \lg n)$ when $\lambda = o(n)$.

In 2018, Miranda and coauthors [95] showed (λ^2) -approximations algorithms for three variants on Sorting by λ -Operations: λ -Reversals, λ -Transpositions, and both. In 2021, Miranda and coauthors [94] introduced the

concept of a λ -permutation, in which every element is up to λ positions from their correct positions. They showed approximation algorithms with factors $O(\lambda^2)$, $O(\lambda)$, and $O(1)$ for Sorting λ -Permutations by λ -Reversals and/or λ -Transpositions.

In 2021, Alexandrino and coauthors [6] also investigated the Sorting by λ -Operations, also using the length-weighted distance, using the cost function ℓ^α , where ℓ is the length of the operation, for some $\alpha \geq 1$. The authors studied five different problems, depending on the representation (signed or unsigned permutations) and the operations (λ -reversal, λ -transposition, or both). They showed $O(\lambda)$ approximation algorithms when $\alpha = 1$, $O(1)$ approximation algorithms when $\alpha \geq 2$ and showed that the five problems are polynomial-time solvable when $\alpha \geq 3$.

7 SORTING BY INTERGENIC OPERATIONS

The majority of representations of genomes focus on mapping the order and orientation of genes. However, in 2016, Biller and coauthors [23, 24] presented studies pointing out that incorporating more genetic structures into models can lead to more realistic results. In particular, intergenic regions were presented as one of the possible structures to be incorporated into the models. The intergenic regions are present between each pair of consecutive genes and at the ends of a linear genome. The size of an intergenic region is the number of nucleotides in it.

In the following sections, we present how the rearrangement events affect a genome in the intergenic context. Furthermore, we overview several works considering three aspects: (i) Genomes sharing the same set of genes; (ii) Genomes with distinct sets of genes; (iii) Genome with multiple copies of a gene.

7.1 Intergenic Operations

To represent a genome considering both the order of genes and the size of the intergenic regions two elements are used: (i) $S^n = (S_1 S_2 \dots S_n)$ represents the sequence of genes. If each gene in the genome is unique, then S is treated as a permutation, otherwise it is treated as a string. Each element S_i receives a '+' or '-' sign to indicate the orientation of the i -th gene. (ii) $\check{S}^{n+1} = (\check{S}_1, \check{S}_2, \dots, \check{S}_{n+1})$ is a list of non-negative integers and represents the intergenic region sizes. The representation is also classified as *signed* or *unsigned* regarding the information about the orientation of the genes. In the following, we describe how intergenic operations affect a genome.

Given a genome representation $(S^n, \check{S}^{n+1})$, an *intergenic reversal*, denoted by $\rho_{(x,y)}^{(i,j)}$, such that $1 \leq i \leq j \leq n$, $\{x, y\} \subset \mathbb{N}_0$, $x \leq \check{S}_i$, and $y \leq \check{S}_{j+1}$, splits the intergenic regions $\check{S}_i$ (into x and $x' = \check{S}_i - x$) and $\check{S}_{j+1}$ (into y and $y' = \check{S}_{j+1} - y$), and inverts the segment $(x', S_i, \check{S}_{i+1}, \dots, \check{S}_j, S_j, y)$ swapping the signs of the elements from S_i to S_j . An intergenic reversal $\rho_{(x,y)}^{(i,j)}$ applied to $(S^n, \check{S}^{n+1})$ results in $(S^{n'}, \check{S}^{n+1'})$, such that $\check{S}'_i = x + y$ and $\check{S}'_{j+1} = x' + y'$:

$$\begin{aligned} S^n &= (S_1 S_2 \dots S_{i-1} S_i S_{i+1} \dots S_{j-1} S_j S_{j+1} \dots S_n) \\ \check{S}^{n+1} &= (\check{S}_1, \check{S}_2, \dots, \check{S}_{i-1}, \check{S}_i, \check{S}_{i+1}, \dots, \check{S}_j, \check{S}_{j+1}, \check{S}_{j+2}, \dots, \check{S}_{n+1}) \\ (S^n, \check{S}^{n+1}) \cdot \rho_{(x,y)}^{(i,j)} &= (S^{n'}, \check{S}^{n+1'}) \\ S^{n'} &= (S_1 S_2 \dots S_{i-1} \underline{-S_j -S_{j-1} \dots -S_{i+1} -S_i} S_{j+1} \dots S_n) \\ \check{S}^{n+1'} &= (\check{S}_1, \check{S}_2, \dots, \check{S}_{i-1}, \underline{\check{S}'_i, \check{S}_j, \dots, \check{S}_{i+1}, \check{S}'_{j+1}}, \check{S}_{j+2}, \dots, \check{S}_{n+1}) \end{aligned}$$

When intergenic reversals are applied to a representation where the orientation of the genes is unknown the signs of the elements from S^n remain omitted.

Given a genome representation $(S^n, \check{S}^{n+1})$, an *intergenic transposition*, denoted by $\tau_{(x,y,z)}^{(i,j,k)}$, such that $1 \leq i < j < k \leq n+1$, $\{x, y, z\} \subset \mathbb{N}_0$, $x \leq \check{S}_i$, $y \leq \check{S}_j$, and $z \leq \check{S}_k$, splits the intergenic regions $\check{S}_i$ (into x and $x' = \check{S}_i - x$), $\check{S}_j$ (into y

and $y' = \check{S}_j - y$, and $\check{S}_k$ (into z and $z' = \check{S}_k - z$), and swaps the positions of the segments $(x', S_i, \check{S}_{i+1}, \dots, \check{S}_{j-1}, S_{j-1}, y)$ and $(y', S_j, \check{S}_{j+1}, \dots, \check{S}_{k-1}, S_{k-1}, z)$. An intergenic transposition $\tau_{(x,y,z)}^{(i,j,k)}$ applied to $(S^n, \check{S}^{n+1})$ results in $(S^{n'}, \check{S}^{n+1'})$, such that $\check{S}'_i = x + y'$, $\check{S}'_{i+k-j} = z + x'$, and $\check{S}'_k = y + z'$:

$$\begin{aligned} S^n &= (S_1 \ S_2 \ \dots \ S_{i-1} \ S_i \ S_{i+1} \ \dots \ S_{j-1} \ S_j \ S_{j+1} \ \dots \ S_{k-1} \ S_k \ S_{k+1} \ \dots \ S_n) \\ \check{S}^{n+1} &= (\check{S}_1, \check{S}_2, \dots, \check{S}_{i-1}, \check{S}_i, \check{S}_{i+1}, \dots, \check{S}_{j-1}, \check{S}_j, \check{S}_{j+1}, \dots, \check{S}_{k-1}, \check{S}_k, \check{S}_{k+1}, \dots, \check{S}_{n+1}) \\ (S^n, \check{S}^{n+1}) \cdot \tau_{(x,y,z)}^{(i,j,k)} &= (S^{n'}, \check{S}^{n+1'}) \\ S^{n'} &= (S_1 \ S_2 \ \dots \ S_{i-1} \ \underline{S_j \ S_{j+1} \ \dots \ S_{k-1}} \ S_i \ S_{i+1} \ \dots \ S_{j-1} \ S_k \ S_{k+1} \ \dots \ S_n) \\ \check{S}^{n+1'} &= (\check{S}_1, \check{S}_2, \dots, \check{S}_{i-1}, \underline{\check{S}'_i, \check{S}'_{j+1}, \dots, \check{S}'_{k-1}}, \check{S}'_{i+k-j}, \check{S}_{i+1}, \dots, \check{S}_{j-1}, \check{S}'_k, \check{S}_{k+1}, \dots, \check{S}_{n+1}) \end{aligned}$$

An *intergenic block-interchange* is a generalization of the intergenic transposition: given a genome representation $(S^n, \check{S}^{n+1})$, an intergenic block-interchange, denoted by $BI_{(v,w,x,y)}^{(i,j,o,p)}$ with $1 \leq i < j \leq o < p \leq n+1$, $\{v, w, x, y\} \subset \mathbb{N}_0$, $v \leq \check{S}_i$, $w \leq \check{S}_j$, $x \leq \check{S}_o$, and $y \leq \check{S}_p$, splits the intergenic regions $\check{S}_i$ (into v and $v' = \check{S}_i - v$), $\check{S}_j$ (into w and $w' = \check{S}_j - w$), $\check{S}_o$ (into x and $x' = \check{S}_o - x$), and $\check{S}_p$ (into y and $y' = \check{S}_p - y$), and swaps the position of the segments $(v', S_i, \check{S}_{i+1}, \dots, \check{S}_{j-1}, S_{j-1}, w)$ and $(x', S_o, \check{S}_{o+1}, \dots, \check{S}_{p-1}, S_{p-1}, y)$. An intergenic block-interchange $BI_{(v,w,x,y)}^{(i,j,o,p)}$ applied to $(S^n, \check{S}^{n+1})$ results in $(S^{n'}, \check{S}^{n+1'})$, such that $\check{S}'_i = v + x'$, $\check{S}'_{i+p-o} = y + w'$, $\check{S}'_{o+p-o-j+i} = x + v'$, and $\check{S}'_p = w + y'$. The distinguishing feature of the intergenic block-interchange event, compared to the intergenic transposition event, is that the affected segments are not necessarily adjacent. However, when $j = o$ the intergenic block-interchange works exactly as an intergenic transposition.

An *intergenic move* is a variation of the intergenic transposition where one of the affected segments is an intergenic region: given a genome representation $(S^n, \check{S}^{n+1})$, an intergenic move, denoted by $\mu_{(x)}^{(i,j)}$ with $1 \leq i \leq n+1$, $1 \leq j \leq n+1$, $x \in \mathbb{N}_0$, and $x \leq \check{S}_i$, moves x nucleotides from the intergenic region $\check{S}_i$ to the intergenic region $\check{S}_j$. An intergenic move $\mu_{(x)}^{(i,j)}$ applied to $(S^n, \check{S}^{n+1})$ results in $(S^{n'}, \check{S}^{n+1'})$, such that $\check{S}'_i = \check{S}_i - x$ and $\check{S}'_j = \check{S}_j + x$.

Given a genome representation $(S^n, \check{S}^{n+1})$, an *intergenic indel*, denoted by $\delta_{(x)}^{(i)}$, such that $1 \leq i \leq n+1$, $x \in \mathbb{N}_0$, and $x \geq -\check{S}_i$, removes x nucleotides from the intergenic region $\check{S}_i$ case $x < 0$. Otherwise, it inserts x nucleotides into the intergenic region $\check{S}_i$. An intergenic indel $\delta_{(x)}^{(i)}$ applied to $(S^n, \check{S}^{n+1})$ results in $(S^{n'}, \check{S}^{n+1'})$, such that $\check{S}'_i = \check{S}_i + x$. Note that these indels affect only intergenic regions. However, there is an extended definition where the indel operation may affect both genes and intergenic regions. In Section 7.3 problems consider this extended definition.

It is worth noting that, when dealing with a rearrangement model composed exclusively of conservative events, the compared genomes must be balanced, which means that they must share the same set of genes, each gene must have the same number of copies, and the total number of nucleotides in the intergenic regions of the source and target genomes must be equal.

7.2 Genomes with Same Set of Genes

In this section, we overview the works in the intergenic context where the compared genomes share the same set of genes. Note that in this scenario the indel event affects only intergenic regions.

In 2016, Bulteau and coauthors [36] presented a polynomial-time algorithm for the Sorting by Intergenic DCJ and Indel Operations problem (SbIDI) on signed instances. However, in 2017, Fertin and coauthors [62] showed

that the Sorting by Intergenic DCJs problem (SbID) on signed instances is NP-hard. The authors also provided a $\frac{4}{3}$ -approximation algorithm for the problem. Both of those works were generalized in the weighed approach of Simonatis and coauthors [115] described in Section 4.4. Their work also allows the operations to be applied in any genome composed of multiple chromosomes, while the original works required the chromosomes to be co-tailed (i.e., have the extremities composed of the same set of genes).

The first work in the intergenic context addressing the reversal and transposition events investigated the restricted version called super short operations (SSOs), where each event can affect at most two genes. In 2019, Oliveira and coauthors [104] investigated the Sorting by Super Short Intergenic Transpositions problem (SbSSIT) on unsigned instances and presented a 3-approximation algorithm. Furthermore, the authors investigated the Sorting by Super Short Intergenic Reversals (SbSSIR) and Sorting by Super Short Intergenic Operations of Reversal and Transposition (SbSSIRT) problems on signed and unsigned instances. For both problems, the authors presented algorithms that guarantee approximation factors of 5 and 3 considering signed and unsigned instances, respectively. The authors also showed better approximations for a class of instances with certain characteristics regarding the number of inversions. The complexities of the three problems remain unknown to this day.

Considering reversal and transposition events without any constraints, Brito and coauthors [28] showed that the Sorting by Intergenic Reversals (SbIR), Sorting by Intergenic Operations of Reversal and Indel (SbIRI), Sorting by Intergenic Operations of Reversal and Transposition (SbIRT), and Sorting by Intergenic Operations of Reversal, Transposition, and Indel (SbIRTI) problems on unsigned instances belong to the NP-hard class. For SbIR and SbIRI problems the authors presented a 4-approximation algorithm, and a $\frac{9}{4}$ -approximation algorithm for the SbIRT and SbIRTI problems. Furthermore, the authors designed approximation algorithms for different cost functions adopting a weighted approach. In 2021, Brito and coauthors [29] presented an improved algorithm for the SbIRT problem that guarantees an approximation factor of 4. Furthermore, the authors presented a 3-approximation algorithm for the Sorting by Intergenic Operations of Reversal, Transposition, and Move problem (SbIRTM) on unsigned instances. In the same year, Oliveira and coauthors [103] investigated the Sorting by Intergenic Transpositions (SbIT) and Sorting by Intergenic Operations of Transposition and Move (SbITM) problems on unsigned instances and presented algorithms with approximation factors of $\frac{7}{2}$ and $\frac{5}{2}$ for the former and latter problems, respectively.

Addressing signed instances, in 2021, Oliveira and coauthors [102] showed that the SbIR problem is NP-hard and presented an approximation algorithm with factor 2. Along with the work, the authors designed a 2-approximation for the SbIRI problem on signed instances. However, the complexity of the SbIRI problem is still unknown. The SbIRT and SbIRTM problems on signed instances are NP-hard and the best approximation factors known to date are 3 and $\frac{5}{2}$, respectively [103]. In 2022, Brito and coauthors [31] showed that the Sorting by Intergenic Operations of Reversal and Move (SbIRM) and Sorting by Intergenic Operations of Reversal, Move and Indel (SbIRMI) problems on signed instances belong to the NP-hard class and presented 2-approximation algorithms for both problems.

Apart from the reversal and transposition events, the block-interchange is another event considered in the intergenic context. In 2019, Dias and coauthors [52] investigated the Sorting by Intergenic Block-Interchange problem (SbIBI) and presented an approximation algorithm with factor 2. However, the complexity of the SbIBI problem remains unknown.

Since intergenic regions, compared to genes, tend to be more susceptible to change, models introduced a degree of flexibility. This feature motivates specifying a range of valid values for the size of each intergenic region of the target genome. In 2021, Brito and coauthors [30] introduced the Sorting by Intergenic Reversals with Flexible Intergenic Regions (SbFIR) and Sorting by Intergenic Operations of Reversal and Transposition with Flexible Intergenic Regions (SbFIRT) problems on unsigned instances. The authors presented a 4-approximation for the former problem and $\frac{9}{2}$ -approximation algorithm for the latter. In 2022, Brito and coauthors [27] investigated the Sorting by Intergenic Transpositions with Flexible Intergenic Regions problem (SbFIT) and presented an

approximation algorithm with a factor of $\frac{7}{2}$. Furthermore, the SbFIR and SbFIRT problems were investigated on signed instances, and algorithms with approximation factors of 2 and 3 were designed for the SbFIR and SbFIRT problems, respectively. However, to date, the complexity of the SbFIR problem remains unknown.

7.3 Genomes with Distinct Sets of Genes

In the intergenic context, non-conservative rearrangement events affect both genes and intergenic regions when the compared genomes have distinct sets of genes. In 2021, Alexandrino and coauthors [9] investigated the Sorting by Intergenic Operations of Reversal and Indel (SbIRI), Sorting by Intergenic Operations of Transposition and Indel (SbITI), and Sorting by Intergenic Operations of Reversal, Transposition, and Indel (SbIRTI) problems on unsigned instances. The study considers that compared genomes may have different gene contents but each gene is unique. The authors showed that all the problems belong to NP-hard class and designed approximation algorithms with factors 4, $\frac{9}{2}$, and 6 for the SbIRI, SbITI, and SbIRTI, respectively. In 2022, Alexandrino and coauthors presented an improved algorithm for the SbITI problem that ensures an approximation factor of 4 [11]. In the same year, Alexandrino and coauthors [4] investigated the SbIRI problem on signed instances and presented a $\frac{5}{2}$ -approximation algorithm. However, to date, the complexity of the problem remains unknown.

7.4 Genomes with Multiples Copies of a Gene

Another important progress in the intergenic context was the possibility to compare genomes with multiple copies of a gene. In this section, we present an overview of works with this characteristic. The models so far focus on conservative events. For this reason, the genomes compared should be balanced, that is, they should have the same set of genes and number of copies. Furthermore, the total number of nucleotides in the intergenic regions of the source and target genomes should be equal.

When the compared genomes have multiple copies of a gene, the most common approach to ensure an approximation is to simplify the genomes with a string partition problem. Originally, relations between partition and distance problems were established for distances involving the events of reversal and transposition [42, 113] in signed and unsigned strings without intergenic regions. In addition to these works using the partition problem name, there is an independently earlier work that approached genomes with multiple copies of a gene with a similar approach but using the name *cover problem* to assign unique labels to the duplicated genes, which was used for computing the Inversion and Indels distance [92]. In 2017, Rubert and coauthors [111] proposed a similar relation for the DCJ Distance problem in signed strings without intergenic regions, and presented a linear-time algorithm that has an approximation factor of $\theta(k)$.

In 2021, Siqueira and coauthors [119] introduced the Intergenic Transposition Distance (ITD), Intergenic Reversal Distance (IRD), and Intergenic Reversal and Transposition Distance (IRTD) problems on unsigned instances. The ITD, IRD, and IRTD problems belong to the NP-hard class. The authors adapted the partition problems to include intergenic regions. Based on a relationship with the Minimum Common Intergenic String Partition and the Reverse Minimum Common Intergenic String Partition problems, the authors designed approximation algorithms for the ITD, IRD, and IRTD problems that ensure a factor of $\theta(k)$, where k is the maximum number of copies of a gene in the genomes. In 2022, Siqueira and coauthors [118] investigated the IRD and IRTD problems on signed instances and showed a similar relationship with the Signed Minimum Common Intergenic String Partition problem. For both problems, the authors also designed a $\theta(k)$ -approximation algorithm, where k is the maximum number of copies of a gene in the genomes. Furthermore, both problems are NP-hard on signed instances.

Beyond approximations, there are FPT algorithms for rearrangement distances on strings that were not yet adapted to include intergenic regions. In 2016, Bulteau and coauthors [33] studied algorithms for the Reversal Distance problem and the Prefix Reversal Distance problem parameterized by the number of blocks in the string

partition and the size of the alphabet (i.e., number of distinct characters in the strings). The authors consider both signed and unsigned strings. In 2021, Lafond and Zhu [84] proposed the Permutation-Constraint Common String Partition problem, which can be used in multiple rearrangement distance problems when the number of copies of each gene is limited to produce algorithms parameterized by the distance.

8 CONCLUSIONS

In this survey, we reviewed the main results of genome rearrangement problems. Many improvements have been made since 2010, especially in showing the complexity of problems with open complexity for more than 10 years. Future works may advance the problems by showing their computational complexity and eventually improve the approximation factor of proposed algorithms.

ACKNOWLEDGMENTS

This work was supported by the National Council of Technological and Scientific Development, CNPq (grants 425340/2016-3 and 202292/2020-7), the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001, and the São Paulo Research Foundation, FAPESP (grants 2013/08293-7, 2015/11937-9, and 2019/27331-3).

REFERENCES

- [1] Max A. Alekseyev. 2008. Multi-Break Rearrangements and Breakpoint Re-Uses: From Circular to Linear Genomes. *Journal of Computational Biology* 15, 8 (2008), 1117–1131.
- [2] Max A Alekseyev and Pavel A Pevzner. 2008. Multi-break Rearrangements and Chromosomal Evolution. *Theoretical Computer Science* 395, 2-3 (2008), 193–202.
- [3] Alexsandro Oliveira Alexandrino, Klairton Lima Brito, Andre Rodrigues Oliveira, Ulisses Dias, and Zanoni Dias. 2022. A 1.375-Approximation Algorithm for Sorting by Transpositions with Faster Running Time. In *Proceedings of the 15th Brazilian Symposium on Bioinformatics (BSB'2022)*. Lecture Notes in Computer Science, Vol. 13523. Springer Nature Switzerland, 147–157.
- [4] Alexsandro Oliveira Alexandrino, Klairton Lima Brito, Andre Rodrigues Oliveira, Ulisses Dias, and Zanoni Dias. 2022. Reversal and Indel Distance with Intergenic Region Information. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 0, PP (2022), 1–13. <https://doi.org/10.1109/TCBB.2022.3215615>
- [5] Alexsandro Oliveira Alexandrino, Carla Negri Lintzmayer, and Zanoni Dias. 2020. Sorting Permutations by Fragmentation-Weighted Operations. *Journal of Bioinformatics and Computational Biology* 18, 2 (2020), 2050006.1–2050006.31.
- [6] Alexsandro Oliveira Alexandrino, Guilherme Henrique Santos Miranda, Carla Negri Lintzmayer, and Zanoni Dias. 2021. Length-weighted λ -rearrangement Distance. *Journal of Combinatorial Optimization* 41, 3 (2021), 579–603.
- [7] Alexsandro Oliveira Alexandrino, Andre Rodrigues Oliveira, Ulisses Dias, and Zanoni Dias. 2020. On the Complexity of Some Variations of Sorting by Transpositions. *Journal of Universal Computer Science* 26, 9 (2020), 1076–1094.
- [8] Alexsandro Oliveira Alexandrino, Andre Rodrigues Oliveira, Ulisses Dias, and Zanoni Dias. 2021. Genome Rearrangement Distance with Reversals, Transpositions, and Indels. *Journal of Computational Biology* 28, 3 (2021), 235–247.
- [9] Alexsandro Oliveira Alexandrino, Andre Rodrigues Oliveira, Ulisses Dias, and Zanoni Dias. 2021. Incorporating Intergenic Regions into Reversal and Transposition Distances with Indels. *Journal of Bioinformatics and Computational Biology* 19, 06 (2021), 2140011.
- [10] Alexsandro Oliveira Alexandrino, Andre Rodrigues Oliveira, Ulisses Dias, and Zanoni Dias. 2022. Labeled Cycle Graph for Transposition and Indel Distance. *Journal of Computational Biology* 29, 03 (2022), 243–256.
- [11] Alexsandro Oliveira Alexandrino, Andre Rodrigues Oliveira, Géraldine Jean, Guillaume Fertin, Ulisses Dias, and Zanoni Dias. 2022. Transposition Distance Considering Intergenic Regions for Unbalanced Genomes. In *Proceedings of the 18th International Symposium on Bioinformatics Research and Applications (ISBRA'2022)*, Vol. 13760. Springer Nature Switzerland, 100–113.
- [12] David A. Bader, Bernard M. E. Moret, and Mi Yan. 2001. A Linear-Time Algorithm for Computing Inversion Distance Between Signed Permutations with an Experimental Study. *Journal of Computational Biology* 8 (2001), 483–491.
- [13] Martin Bader and Enno Ohlebusch. 2007. Sorting by Weighted Reversals, Transpositions, and Inverted Transpositions. *Journal of Computational Biology* 14, 5 (2007), 615–636.
- [14] Vineet Bafna and Pavel A. Pevzner. 1996. Genome Rearrangements and Sorting by Reversals. *SIAM Journal on Computing* 25, 2 (1996), 272–289.
- [15] Vineet Bafna and Pavel A. Pevzner. 1998. Sorting by Transpositions. *SIAM Journal on Discrete Mathematics* 11, 2 (1998), 224–240.

- [16] Michael A. Bender, Dongdong Ge, Simai He, Haodong Hu, Ron Y. Pinter, Steven S. Skiena, and Firas Swidan. 2008. Improved Bounds on Sorting by Length-Weighted Reversals. *Journal of Computer and System Sciences* 74, 5 (2008), 744–774.
- [17] Anne Bergeron. 2005. A Very Elementary Presentation of the Hannenhalli-Pevzner Theory. *Discrete Applied Mathematics* 146, 2 (2005), 134–145.
- [18] Anne Bergeron, Julia Mixtacki, and Jens Stoye. 2006. A Unifying View of Genome Rearrangements. In *International Workshop on Algorithms in Bioinformatics*. Springer, Springer, Berlin, Heidelberg, 163–173.
- [19] Anne Bergeron, Julia Mixtacki, and Jens Stoye. 2009. A new linear time algorithm to compute the genomic distance via the double cut and join distance. *Theoretical Computer Science* 410, 51 (2009), 5300–5316. <https://doi.org/10.1016/j.tcs.2009.09.008>
- [20] Piotr Berman, Sridhar Hannenhalli, and Marek Karpinski. 2002. 1.375-Approximation Algorithm for Sorting by Reversals. In *Proceedings of the 10th Annual European Symposium on Algorithms (ESA'2002)*, R. Möhring and R. Raman (Eds.). Lecture Notes in Computer Science, Vol. 2461. Springer-Verlag Berlin Heidelberg New York, Berlin/Heidelberg, Germany, 200–210.
- [21] Sangeeta Bhatia, Pedro Feijão, and Andrew R. Francis. 2018. Position and Content Paradigms in Genome Rearrangements: The Wild and Crazy World of Permutations in Genomics. *Bulletin of Mathematical Biology* 80, 12 (2018), 3227–3246.
- [22] Daniel Bienstock and Oktay Günlük. 1994. A degree sequence problem related to network design. *Networks* 24 (1994), 195–205.
- [23] Priscilla Biller, Laurent Guéguen, Carole Knibbe, and Eric Tannier. 2016. Breaking Good: Accounting for Fragility of Genomic Regions in Rearrangement Distance Estimation. *Genome Biology and Evolution* 8, 5 (2016), 1427–1439.
- [24] Priscilla Biller, Carole Knibbe, Guillaume Beslon, and Eric Tannier. 2016. Comparative Genomics on Artificial Life. In *Pursuit of the Universal*. Springer International Publishing, 35–44.
- [25] Mathieu Blanchette, Takashi Kunisawa, and David Sankoff. 1996. Parametric Genome Rearrangement. *Gene* 172, 1 (1996), GC11–GC17.
- [26] Marília DV Braga, Eyla Willing, and Jens Stoye. 2011. Double Cut and Join with Insertions and Deletions. *Journal of Computational Biology* 18, 9 (2011), 1167–1184.
- [27] Klairton Lima Brito, Alessandro Oliveira Alexandrino, Andre Rodrigues Oliveira, Ulisses Dias, and Zanoni Dias. 2022. Genome Rearrangement Distance with a Flexible Intergenic Regions Aspect. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* PP, 0 (2022), 1–13. <https://doi.org/10.1109/TCBB.2022.3165443>
- [28] Klairton Lima Brito, Géraldine Jean, Guillaume Fertin, Andre Rodrigues Oliveira, Ulisses Dias, and Zanoni Dias. 2020. Sorting by Genome Rearrangements on both Gene Order and Intergenic Sizes. *Journal of Computational Biology* 27, 2 (2020), 156–174.
- [29] Klairton Lima Brito, Andre Rodrigues Oliveira, Alessandro Oliveira Alexandrino, Ulisses Dias, and Zanoni Dias. 2021. An Improved Approximation Algorithm for the Reversal and Transposition Distance Considering Gene Order and Intergenic Sizes. *Algorithms for Molecular Biology* 16, 1 (2021), 1–21.
- [30] Klairton Lima Brito, Andre Rodrigues Oliveira, Alessandro Oliveira Alexandrino, Ulisses Dias, and Zanoni Dias. 2021. Reversal and Transposition Distance of Genomes Considering Flexible Intergenic Regions. In *Proceedings of the XI Latin and American Algorithms, Graphs and Optimization Symposium (LAGOS'2021)*. Procedia Computer Science, Elsevier, 21–29.
- [31] Klairton Lima Brito, Andre Rodrigues Oliveira, Alessandro Oliveira Alexandrino, Ulisses Dias, and Zanoni Dias. 2022. A New Approach for the Reversal Distance with Indels and Moves in Intergenic Regions. In *Proceedings of 19th Annual Satellite Conference of RECOMB on Comparative Genomics (RECOMB-CG'2022)*, Vol. 13234. Springer International Publishing, 205–220.
- [32] Laurent Bulteau, Guillaume Fertin, Géraldine Jean, and Christian Komusiewicz. 2021. Sorting by Multi-Cut Rearrangements. *Algorithms* 14, 6 (2021), 169.
- [33] Laurent Bulteau, Guillaume Fertin, and Christian Komusiewicz. 2016. (Prefix) reversal distance for (signed) strings with few blocks or small alphabets. *Journal of Discrete Algorithms* 37 (2016), 44–55.
- [34] Laurent Bulteau, Guillaume Fertin, and Irena Rusu. 2012. Sorting by Transpositions is Difficult. *SIAM Journal on Discrete Mathematics* 26, 3 (2012), 1148–1180.
- [35] Laurent Bulteau, Guillaume Fertin, and Irena Rusu. 2015. Pancake Flipping is hard. *J. Comput. System Sci.* 81, 8 (Dec. 2015), 1556–1574.
- [36] Laurent Bulteau, Guillaume Fertin, and Eric Tannier. 2016. Genome Rearrangements with Indels in Intergenes Restrict the Scenario Space. *BMC Bioinformatics* 17, 14 (2016), 426.
- [37] Laurent Bulteau and Mathias Weller. 2019. Parameterized Algorithms in Bioinformatics: An Overview. *Algorithms* 12, 12 (2019), 256.
- [38] Ben Cameron, Joe Sawada, Wei Therese, and Aaron Williams. 2022. Hamiltonicity of k-Sided Pancake Networks with Fixed-Spin: Efficient Generation, Ranking, and Optimality. *Algorithmica* 0, PP (2022), 1–28. <https://doi.org/10.1007/s00453-022-01022-x>
- [39] Alberto Caprara. 1999. Sorting Permutations by Reversals and Eulerian Cycle Decompositions. *SIAM Journal on Discrete Mathematics* 12, 1 (1999), 91–110.
- [40] Xin Chen. 2010. On Sorting Permutations by Double-Cut-and-Joins. In *Proceedings of the 16th International Computing and Combinatorics Conference (COCOON'2010) (Lecture Notes in Computer Science, Vol. 6196)*. Springer, Berlin, Heidelberg, 439–448.
- [41] Xin Chen. 2013. On Sorting Unsigned Permutations by Double-Cut-and-Joins. *Journal of Combinatorial Optimization* 25, 3 (2013), 339–351.
- [42] Xin Chen, Jie Zheng, Zheng Fu, Peng Nan, Yang Zhong, Stefano Lonardi, and Tao Jiang. 2005. Assignment of Orthologous Genes via Genome Rearrangement. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 2, 4 (2005), 302–315.

- [43] Bhadrachalam Chitturi. 2015. Tighter Upper Bound for Sorting Permutations with Prefix Transpositions. *Theoretical Computer Science* 602 (2015), 22–31.
- [44] David A. Christie. 1996. Sorting Permutations by Block-Interchanges. *Information Processing Letters* 60, 4 (1996), 165–169.
- [45] David A. Christie. 1998. A 3/2-Approximation Algorithm for Sorting by Reversals. In *Proceedings of the 9th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA'1998)*, H. Karloff (Ed.). Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 244–252.
- [46] Josef Cibulka. 2011. On average and highest number of flips in pancake sorting. *Theoretical Computer Science* 412, 8–10 (2011), 822–834.
- [47] David S. Cohen and Manuel Blum. 1995. On the Problem of Sorting Burnt Pancakes. *Discrete Applied Mathematics* 61, 2 (1995), 105–120.
- [48] Luís Felipe I. Cunha, Luis Antonio B. Kowada, Rodrigo de A. Hausen, and Celina M.H. de Figueiredo. 2015. A Faster 1.375-Approximation Algorithm for Sorting by Transpositions. *Journal of Computational Biology* 22, 11 (2015), 1044–1056.
- [49] Thiago da S. Arruda, Ulisses Dias, and Zanoni Dias. 2018. A GRASP-Based Heuristic for the Sorting by Length-Weighted Inversions Problem. *IEEE-ACM Transactions on Computational Biology and Bioinformatics* 15 (2018), 352–363.
- [50] Daniel A. Dalevi, Niklas Eriksen, Kimmo Eriksson, and Siv G. E. Andersson. 2002. Measuring Genome Divergence in Bacteria: A Case Study Using Chlamydian Data. *Journal of Molecular Evolution* 55, 1 (2002), 24–36.
- [51] Ulisses Dias and Zanoni Dias. 2013. Heuristics for the Transposition Distance Problem. *Journal of Bioinformatics and Computational Biology* 11, 5 (2013), 17.
- [52] Ulisses Dias, Andre Rodrigues Oliveira, Klairton Lima Brito, and Zanoni Dias. 2019. Block-Interchange Distance Considering Intergenic Regions. In *Proceedings of the 12th Brazilian Symposium on Bioinformatics (BSB'2019)*. Vol. 11347. Springer International Publishing, 58–69.
- [53] Zanoni Dias and Ulisses Dias. 2015. Sorting by Prefix Reversals and Prefix Transpositions. *Discrete Applied Mathematics* 181 (2015), 78–89.
- [54] Zanoni Dias and João Meidanis. 2002. Sorting by Prefix Transpositions. In *Proceedings of the 9th International Symposium on String Processing and Information Retrieval (SPIRE'2002)*, A. H. F. Laender and A. L. Oliveira (Eds.). Lecture Notes in Computer Science, Vol. 2476. Springer-Verlag Berlin Heidelberg New York, Berlin/Heidelberg, Germany, 65–76.
- [55] Harry Dweighter. 1975. Problem E2569. *American Mathematical Monthly* 82 (1975), 1010.
- [56] Nadia El-Mabrouk. 2000. Genome Rearrangement by Reversals and Insertions/Deletions of Contiguous Segments. In *Annual Symposium on Combinatorial Pattern Matching (CPM'2000)*. Springer, Springer, Berlin, Heidelberg, 222–234.
- [57] Isaac Elias and Tzvika Hartman. 2006. A 1.375-Approximation Algorithm for Sorting by Transpositions. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 3, 4 (2006), 369–379.
- [58] Niklas Eriksen. 2002. $(1+\epsilon)$ -Approximation of Sorting by Reversals and Transpositions. *Theoretical Computer Science* 289, 1 (2002), 517–529.
- [59] Pedro Feijao and Joao Meidanis. 2011. SCJ: A Breakpoint-Like Distance that Simplifies Several Rearrangement Problems. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 8, 5 (2011), 1318–1329. <https://doi.org/10.1109/TCBB.2011.34>
- [60] Guillaume Fertin, Loïc Jankowiak, and Géraldine Jean. 2018. Prefix and suffix reversals on strings. *Discrete Applied Mathematics* 246 (2018), 140–153. <https://doi.org/10.1016/j.dam.2017.07.031>
- [61] Guillaume Fertin, Géraldine Jean, and Anthony Labarre. 2022. Sorting Genomes by Prefix Double-Cut-and-Joins. In *String Processing and Information Retrieval (SPIRE'2022)*. Springer International Publishing, Concepción, Chile, 178–190.
- [62] Guillaume Fertin, Géraldine Jean, and Eric Tannier. 2017. Algorithms for Computing the Double Cut and Join Distance on both Gene Order and Intergenic Sizes. *Algorithms for Molecular Biology* 12, 1 (2017), 16.
- [63] Guillaume Fertin, Anthony Labarre, Irena Rusa, Éric Tannier, and Stéphane Vialette. 2009. *Combinatorics of Genome Rearrangements*. The MIT Press, London, England.
- [64] Johannes Fischer and Simon W. Ginzinger. 2005. A 2-Approximation Algorithm for Sorting by Prefix Reversals. In *Proceedings of the 13th Annual European Conference on Algorithms (ESA'2005) (Lecture Notes in Computer Science, Vol. 3669)*, G. S. Brodal and S. Leonardi (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 415–425.
- [65] Gustavo R. Galvão, Orlando Lee, and Zanoni Dias. 2015. Sorting Signed Permutations by Short Operations. *Algorithms for Molecular Biology* 10, 1 (2015), 1–17.
- [66] Gustavo R. Galvão, Christian Baudet, and Zanoni Dias. 2017. Sorting Circular Permutations by Super Short Reversals. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 14, 3 (2017), 620–633.
- [67] William H. Gates and Christos H. Papadimitriou. 1979. Bounds for sorting by prefix reversal. *Discrete Mathematics* 27, 1 (1979), 47–57.
- [68] Qian-Ping Gu, Shietung Peng, and Ivan H. Sudborough. 1999. A 2-Approximation Algorithm for Genome Rearrangements by Reversals and Transpositions. *Theoretical Computer Science* 210, 2 (1999), 327–339.
- [69] Sridhar Hannenhalli and Pavel A. Pevzner. 1999. Transforming Cabbage into Turnip: Polynomial Algorithm for Sorting Signed Permutations by Reversals. *Journal of the ACM* 46, 1 (1999), 1–27.
- [70] Tzvika Hartman and Roded Sharan. 2005. A 1.5-Approximation Algorithm for Sorting by Transpositions and Transreversals. *Journal of Computer and System Sciences* 70, 3 (2005), 300–320.

- [71] Masud Hasan, Atif Rahman, Md. K. Rahman, Mohammad S. Rahman, Mahfuza Sharmin, and Rukhsana Yeasmin. 2015. Pancake Flipping and Sorting Permutations. *Journal of Discrete Algorithms* 33 (2015), 139–149.
- [72] Lenwood S. Heath and John Paul C. Vergara. 2000. Sorting by Short Block-Moves. *Algorithmica* 28, 3 (2000), 323–352.
- [73] Lenwood S. Heath and John Paul C. Vergara. 2003. Sorting by Short Swaps. *Journal of Computational Biology* 10, 5 (2003), 775–789.
- [74] Andrew J Holland and Don W Cleveland. 2012. Chromoanagenesis and cancer: mechanisms and consequences of localized, complex chromosomal rearrangements. *Nature Medicine* 18, 11 (2012), 1630–1638.
- [75] Mark R. Jerrum. 1985. The Complexity of Finding Minimum-length Generator Sequences. *Theoretical Computer Science* 36, 2-3 (1985), 265–289.
- [76] Haitao Jiang, Haodi Feng, and Daming Zhu. 2014. An 5/4-Approximation Algorithm for Sorting Permutations by Short Block Moves. In *Proceedings of the 25th International Symposium on Algorithms and Computation (ISAAC'2014) (Lecture Notes in Computer Science, Vol. 8889)*, H. Ahn and C. Shin (Eds.). Springer International Publishing, Jeonju, Korea, 491–503.
- [77] Haitao Jiang, Daming Zhu, and Binhai Zhu. 2012. A (1+ ϵ)-Approximation Algorithm for Sorting by Short Block-Moves. *Theoretical Computer Science* 437 (2012), 1–8.
- [78] M.P. Justan, F.P. Muga, and I.H. Sudborough. 2002. On the generalization of the pancake network. In *Proceedings International Symposium on Parallel Architectures, Algorithms and Networks. (I-SPAN'2002)*. IEEE Computer Society, Makati City, Metro Manila, Philippines, 173–178.
- [79] John D. Kececioglu and David Sankoff. 1995. Exact and Approximation Algorithms for Sorting by Reversals, with Application to Genome Rearrangement. *Algorithmica* 13 (1995), 180–210.
- [80] Donald E. Knuth. 1973. *Fundamental Algorithms: The art of Computer Programming*. Addison-Wesley, Reading, Massachusetts.
- [81] Anthony Labarre. 2013. Lower Bounding Edit Distances between Permutations. *SIAM Journal on Discrete Mathematics* 27, 3 (2013), 1410–1428.
- [82] Anthony Labarre. 2020. Sorting by Prefix Block-Interchanges. In *31st International Symposium on Algorithms and Computation (ISAAC'2020)*, Vol. 181. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Hong Kong (Virtual Conference), 55:1–55:15.
- [83] Anthony Labarre and Josef Cibulka. 2011. Polynomial-time sortable stacks of burnt pancakes. *Theoretical Computer Science* 412, 8-10 (2011), 695–702.
- [84] Manuel Lafond and Binhai Zhu. 2021. Permutation-Constrained Common String Partitions with Applications. In *String Processing and Information Retrieval (SPIRE'2021)*, Thierry Lecroq and Hélène Touzet (Eds.). Springer International Publishing, 47–60.
- [85] S. Lakshmivarahan, Jung-Sing Jwo, and Sudarshan Dhall. 1993. Symmetry in Interconnection Networks Based on Cayley Graphs of Permutation Groups: a Survey. *Parallel Computing* 19, 4 (1993), 361–407.
- [86] Jean-François Lefebvre, Nadia El-Mabrouk, Elisabeth R. M. Tillier, and David Sankoff. 2003. Detection and validation of single gene inversions. *Bioinformatics* 19, 1 (2003), i190–i196.
- [87] Guo-Hui Lin and Guoliang Xue. 2001. Signed Genome Rearrangement by Reversals and Transpositions: Models and Approximations. *Theoretical Computer Science* 259, 1-2 (2001), 513–531.
- [88] Ying C. Lin, Chun-Yuan Lin, and Chunhung R. Lin. 2009. Sorting by Reversals and Block-Interchanges with Various Weight Assignments. *BMC Bioinformatics* 10, 1 (2009), 398.
- [89] Carla N. Lintzmayer, Guillaume Fertin, and Zanoni Dias. 2015. Approximation Algorithms for Sorting by Length-Weighted Prefix and Suffix Operations. *Theoretical Computer Science* 593 (2015), 26–41.
- [90] Carla Negri Lintzmayer, Guillaume Fertin, and Zanoni Dias. 2017. Sorting Permutations by Prefix and Suffix Rearrangements. *Journal of Bioinformatics and Computational Biology* 15, 1 (2017), 1750002.
- [91] Aniket C. Mane, Manuel Lafond, Pedro C. Feijao, and Cedric Chauve. 2020. The Distance and Median Problems in the Single-Cut-or-Join Model with Single-Gene Duplications. *Algorithms for Molecular Biology* 15, 1 (2020), 1–14.
- [92] Mark Marron, Krister M. Swenson, and Bernard M.E. Moret. 2004. Genomic Distances under Deletions and Insertions. *Theoretical Computer Science* 325, 3 (2004), 347–860.
- [93] István Miklós, Sándor Z. Kiss, and Eric Tannier. 2014. Counting and sampling SCJ small parsimony solutions. *Theoretical Computer Science* 552 (2014), 83–98. <https://doi.org/10.1016/j.tcs.2014.07.027>
- [94] Guilherme Henrique Santos Miranda, Alessandro Oliveira Alexandrino, Carla Negri Lintzmayer, and Zanoni Dias. 2021. Approximation Algorithms for Sorting λ -Permutations by λ -Operations. *Algorithms* 14, 6 (2021), 175.
- [95] Guilherme Henrique Santos Miranda, Carla Negri Lintzmayer, and Zanoni Dias. 2018. Sorting Permutations by Limited-Size Operations. In *Proceedings of the 5th International Conference on Algorithms for Computational Biology (AlCoB'2018)*. Vol. 10849. Springer International Publishing, Heidelberg, Germany, 76–87.
- [96] Thach C. Nguyen, Hieu T. Ngo, and Nguyen B. Nguyen. 2005. Sorting by Restricted-Length-Weighted Reversals. *Genomics Proteomics & Bioinformatics* 3, 2 (2005), 120–127.
- [97] Andre Rodrigues Oliveira, Alessandro Oliveira Alexandrino, Géraldine Jean, Guillaume Fertin, Ulisses Dias, and Zanoni Dias. 2023. Approximation algorithms for sorting by k-cuts on signed permutations. *Journal of Combinatorial Optimization* 45, 1 (2023), 6.

- [98] Andre Rodrigues Oliveira, Klairton Lima Brito, Ulisses Dias, and Zanoni Dias. 2019. On the Complexity of Sorting by Reversals and Transpositions Problems. *Journal of Computational Biology* 26, 11 (2019), 1223–1229.
- [99] Andre Rodrigues Oliveira, Klairton Lima Brito, Zanoni Dias, and Ulisses Dias. 2019. Sorting by Weighted Reversals and Transpositions. *Journal of Computational Biology* 26 (2019), 420–431.
- [100] Andre Rodrigues Oliveira, Ulisses Dias, and Zanoni Dias. 2017. On the Sorting by Reversals and Transpositions Problem. *Journal of Universal Computer Science* 23 (2017), 868–906.
- [101] Andre Rodrigues Oliveira, Guillaume Fertin, Ulisses Dias, and Zanoni Dias. 2018. Sorting Signed Circular Permutations by Super Short Operations. *Algorithms for Molecular Biology* 13, 1 (2018), 13.
- [102] Andre Rodrigues Oliveira, Géraldine Jean, Guillaume Fertin, Klairton Lima Brito, Laurent Bulteau, Ulisses Dias, and Zanoni Dias. 2021. Sorting Signed Permutations by Intergenic Reversals. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 18, 6 (2021), 2870–2876.
- [103] Andre Rodrigues Oliveira, Géraldine Jean, Guillaume Fertin, Klairton Lima Brito, Ulisses Dias, and Zanoni Dias. 2021. Sorting Permutations by Intergenic Operations. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 18, 6 (2021), 2080–2093.
- [104] Andre Rodrigues Oliveira, Géraldine Jean, Guillaume Fertin, Ulisses Dias, and Zanoni Dias. 2019. Super Short Operations on Both Gene Order and Intergenic Sizes. *Algorithms for Molecular Biology* 14, 1 (2019), 1–17.
- [105] Jayakumar Pai and Bhadrachalam Chitturi. 2021. Approximation algorithms for sorting permutations by extreme block-interchanges. *Theoretical Computer Science* 891 (2021), 105–115. <https://doi.org/10.1016/j.tcs.2021.08.031>
- [106] Franck Pellestor and Vincent Gatinois. 2020. Chromoanagenesis: a piece of the macroevolution scenario. *Molecular Cytogenetics* 13, 1 (2020), 1–9.
- [107] Pavel A. Pevzner and Michael S. Waterman. 1995. Open Combinatorial Problems in Computational Molecular Biology. In *Proceedings of the 3rd Israel Symposium on the Theory of Computing Systems (ISTCS'1995)*. IEEE Computer Society, Washington, DC, USA, 158–173.
- [108] Ron Y. Pinter and Steven Skiena. 2002. Genomic Sorting with Length-Weighted Reversals. *Genome Informatics* 13 (2002), 103–111.
- [109] Xingqin Qi, Jichang Wu, Shuguang Li, and Guojun Li. 2008. Sorting by transpositions: dealing with length-weighted models. *International Journal of Bioinformatics Research and Applications* 4, 2 (2008), 164.
- [110] Atif Rahman, Swakkhar Shatabda, and Masud Hasan. 2008. An Approximation Algorithm for Sorting by Reversals and Transpositions. *Journal of Discrete Algorithms* 6, 3 (2008), 449–457.
- [111] Diego P. Rubert, Pedro Feijão, Marília Dias Vieira Braga, Jens Stoye, and Fábio Henrique Viduani Martinez. 2017. Approximating the DCJ Distance of Balanced Genomes in Linear Time. *Algorithms for Molecular Biology* 12, 1 (2017), 1–13.
- [112] Cathal Seoighe, Nancy Federspiel, Ted Jones, Nancy Hansen, Vesna Bivolarovic, Ray Surzycki, Raquel Tamse, Caridad Komp, Lucas Huizar, Ronald W. Davis, Stewart Scherer, Evelyn Tait, Duncan J. Shaw, David Harris, Lee Murphy, Karen Oliver, Kate Taylor, Marie-Adele Rajandream, Bart G. Barrell, and Kenneth H. Wolfe. 2000. Prevalence of small inversions in yeast gene order evolution. *Proceedings of the National Academy of Sciences* 97, 26 (2000), 14433–14437.
- [113] Dana Shapira and James A. Storer. 2007. Edit distance with move operations. *Journal of Discrete Algorithms* 5, 2 (2007), 380–392.
- [114] Luiz Augusto G. Silva, Luis Antonio B. Kowada, Norai Romeu Rocco, and Maria Emilia M. T. Walter. 2022. A new 1.375-approximation algorithm for sorting by transpositions. *Algorithms for Molecular Biology* 17, 1 (2022), 1–17.
- [115] Pijus Simonaitis, Annie Chateau, and Krister M. Swenson. 2018. A General Framework for Genome Rearrangement with Biological Constraints. In *Comparative Genomics*. Springer International Publishing, 49–71.
- [116] Pijus Simonaitis, Annie Chateau, and Krister M. Swenson. 2019. Weighted Minimum-Length Rearrangement Scenarios. In *19th International Workshop on Algorithms in Bioinformatics (WABI'2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 143)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 13:1–13:17.
- [117] Pijus Simonaitis and Benjamin J. Raphael. 2022. A Maximum Parsimony Principle for Multichromosomal Complex Genome Rearrangements. In *22nd International Workshop on Algorithms in Bioinformatics (WABI'2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 242)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 21:1–21:22.
- [118] Gabriel Siqueira, Alexsandro Oliveira Alexandrino, and Zanoni Dias. 2022. Signed Rearrangement Distances Considering Repeated Genes and Intergenic Regions. In *Proceedings of 14th International Conference on Bioinformatics and Computational Biology (BICoB'2022)*, Vol. 83. EasyChair, 31–42.
- [119] Gabriel Siqueira, Alexsandro Oliveira Alexandrino, Andre Rodrigues Oliveira, and Zanoni Dias. 2021. Approximation Algorithm for Rearrangement Distances Considering Repeated Genes and Intergenic Regions. *Algorithms for Molecular Biology* 16, 1 (2021), 1–23.
- [120] Philip Stephens, Chris Greenman, Beiyuan Fu, Fengtang Yang, Graham Bignell, Laura Mudie, Erin Pleasance, King Lau, David Beare, Lucy Stebbings, Stuart McLaren, Meng-Lay Lin, David McBride, Ignacio Varela, Serena Nik-Zainal, Catherine Leroy, Mingming Jia, Andrew Menzies, Adam Butler, and Peter Campbell. 2011. Massive Genomic Rearrangement Acquired in a Single Catastrophic Event during Cancer Development. *Cell* 144 (01 2011), 27–40. <https://doi.org/10.1016/j.cell.2010.11.055>
- [121] Chengcheng Sun and Haitao Jiang. 2021. An approximation algorithm for unifying adjacencies by double cut and joins in unsigned genomes. In *2021 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, 86–91. <https://doi.org/10.1109/BIBM52615.2021.9669334>

- [122] Krister Swenson, Pijus Simonaitis, and Mathieu Blanchette. 2016. Models and algorithms for genome rearrangement with positional constraints. *Algorithms for Molecular Biology* 11 (05 2016), 13. <https://doi.org/10.1186/s13015-016-0065-9>
- [123] Krister M. Swenson and Pijus Simonaitis. 2018. Finding local genome rearrangements. *Algorithms for Molecular Biology* 13 (2018), 9–23.
- [124] Firas Swidan, Michael A. Bender, Dongdong Ge, Simai He, Haodong Hu, and Ron Y. Pinter. 2004. Sorting by Length-Weighted Reversals: Dealing with Signs and Circularity. In *Proceedings of the 15th Annual Symposium on Combinatorial Pattern Matching (CPM'2004)*, S. Sahinalp, S. Muthukrishnan, and U. Dogrusoz (Eds.). Lecture Notes in Computer Science, Vol. 3109. Springer Berlin Heidelberg, Heidelberg, Germany, 32–46.
- [125] Eric Tannier and Marie-France Sagot. 2004. Sorting by Reversals in Subquadratic Time. In *Proceedings of the 15th Annual Symposium on Combinatorial Pattern Matching (CPM'2004) (Lecture Notes in Computer Science, Vol. 3109)*, S. C. Sahinalp, S. Muthukrishnan, and U. Dogrusoz (Eds.). Springer Berlin Heidelberg, Berlin/Heidelberg, Germany, 1–13.
- [126] John P. C. Vergara. 1998. *Sorting by Bounded Permutations*. Ph. D. Dissertation. Virginia Polytechnic Institute and State University.
- [127] Maria E. M. T. Walter, Zaroni Dias, and João Meidanis. 1998. Reversal and Transposition Distance of Linear Chromosomes. In *Proceedings of the 5th International Symposium on String Processing and Information Retrieval (SPIRE'1998)*. IEEE Computer Society, Los Alamitos, CA, USA, 96–102.
- [128] Eyla Willing, Jens Stoye, and Marília Braga. 2021. Computing the Inversion-Indel Distance. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 18, 6 (2021), 2314–2326.
- [129] Eyla Willing, Simone Zaccaria, Marília DV Braga, and Jens Stoye. 2013. On the Inversion-Indel Distance. *BMC Bioinformatics* 14 (2013), S3.
- [130] Sophia Yancopoulos, Oliver Attie, and Richard Friedberg. 2005. Efficient Sorting of Genomic Permutations by Translocation, Inversion and Block Interchange. *Bioinformatics* 21, 16 (2005), 3340–3346.
- [131] Ron Zeira and Ron Shamir. 2019. Genome Rearrangement Problems with Single and Multiple Gene Copies: A Review. In *Bioinformatics and Phylogenetics*. Springer International Publishing, 205–241.
- [132] Shu Zhang, Daming Zhu, Haitao Jiang, Jiong Guo, Haodi Feng, and Xiaowen Liu. 2021. Sorting a Permutation by Best Short Swaps. *Algorithmica* 83, 7 (2021), 1953–1979.